

Telegram Bot & Kurslar Platformasi — Texnik Audit va Arxitektura Hujjati

Mustaqil Texnik Audit va AI Tahlili uchun to'liq tushuntirish va verifikatsiya hisoboti

ARXITEKTURA V2.0 (SALES + EPHEMERAL)

BARCHA 5 SHART 100% YOPILDI

PRODUCTION READY

1. TIZIM ARXITEKTURASI VA VAZIFALAR TAQSIMOTI

Aniq Demarkatsiya

Kontent xavfsizligini 100% kafolatlash va veb-ekranlar orqali videolarni o'g'irlashni oldini olish maqsadida vazifalar qat'iy ajratilgan:

1. Mini App — Faqat Reklama va Xaridlar (Sales Funnel)

- Kurslar Vitrinasi va Reklamasi:** Muqova, chegirmalar, o'qituvchi profili, natijalar, galereya va tez-tez so'raladigan savollar.
- Dastur Sillabusi:** Talaba nimalar o'rganishini ko'radi. Qulflangan darslar to'lovga, xarid qilinganlar guruhga yo'naltiradi.
- To'lov mexanizmi:** Payme, Click, karta chekini yuklash, promokodlar va hamyon balans.
- Nol Veb-Player:** Mini App ichida hech qanday video pleyer mavjud emas. Video fayllar vebga chiqmaydi.

2. Telegram Yopiq Guruhi — Darslarni Ko'rish va Ta'lim

- Faqat Ephemeral Yetkazish:** Dars videolari hech qachon guruhga ommaviy tashlanmaydi, faqat `send_ephemeral_video` orqali ochiladi.
- Faqat Bosgan Foydalanuvchiga Ko'rinadi:** `receiver_user_id` va `callback_query_id` bilan Telegram serveri faqat bosgan talabaga ko'rsatadi.
- Xaridorga Bog'langan Guruh:** Guruhga faqat bir martalik tasdiqlangan havola (`chat_join_request`) orqali kiriladi.
- Dars Huquqi Guruhdan Mustaqil:** Guruh a'zosi bo'lsa ham, har bir dars o'zining `course_id` ruxsatiga ega bo'lmasa ochilmaydi.

2. AUDITOR MEZONLARI VA XAVFSIZLIK MEKANIZMLARI

10/10 Xavfsizlik

1. Shaxsiy Chatda Xom Video Jo'natish To'liq To'xtatildi (Anti-AyuGram / Anti-Piracy)

Muammo: AyuGram kabi modifikatsiyalangan klientlar shaxsiy chatga `protect_content=True` bilan yuborilgan har qanday videoni ham to'g'ridan-to'g'ri diskka yuklab oladi.
Yechim (`bot_service.py: deliver_private_lesson`): Bot shaxsiy chatda pullik dars so'ralganda **hech qachon xom video yubormaydi**. Buning o'rniga Mini App (katalog) va Yopiq O'quv Guruhiga yo'naltiruvchi interaktiv kartochka jo'natiladi. Faqat adminlar yoki bepul preview darslar bundan mustasno.

2. Guruhda Ephemeral Yetkazishda Fallback Yo'q, protect_content=True va HTML Parse Mode

Yechim (`bot_service.py: send_ephemeral_video` & `handle_student_ephemeral_lesson`): Guruhda dars tugmasi bosilganda faqat `send_ephemeral_video` ishlatiladi (`parse_mode="HTML"`, hardcoded `protect_content=True`). Agar Telegram API xatolik qaytarsa (429, 403 yoki eski klient), **shaxsiy chatga hech qanday video tushib qolmaydi**. `delivery_error_text` orqali talabaga aniq diagnostika ko'rsatiladi.

3. Join-Request Failure Recovery va Bir Martalik Havola Auditi

Yechim (`bot_service.py: handle_chat_join_request` & `purchases.py: revoke_join_request_link`): Havola faqat haqiqiy xaridorning Telegram ID'siga bog'lanadi. Begona odam kirs, zayavkasi rad etiladi, haqiqiy xaridorning havolasi **kuyib ketmaydi**. `approveChatJoinRequest` chaqirilganda, faqat `result["ok"] == True` bo'lsagina havola bekor qilinadi (`revokeChatInviteLink`) va bazada `invite_consumed_at = now()` qayd etiladi. Decline natijasi ham to'liq tekshiriladi.

4. Enrollment Exclusivity & Strict Booleans (H1 & H2 Yechimlari)

H1 Yechimi: Bekor qilingan enrollment hech qachon eski xarid orqali avtomatik qayta tiklanmaydi (resurrection xatosi olib tashlandi). Darsga kirish faqat faol (`status == "active"`) enrollment orqali beriladi.
H2 Yechimi: `flag_is_true()` funksiyasi joriy qilindi — `"false"`, `"0"` yoki bo'sh stringlar hech qachon True deb hisoblanmaydi. Kurs va darsning `published` holati qat'iy tekshiriladi (H5).

5. Fail-Closed Himoya, Asinxron Loop va Admin Sessiyalari (H3, H4, E, F)

H3: DB xatolarida blacklist fail-closed rejimda ishlaydi — nozik amallar bloklanadi.
H4: Polling loop (`asyncio.create_task(_safe_handle_tg_update)`) orqali non-blocking concurrent ishlaydi.
E & F: Admin videoni 1-bosishda saqlash faqat matching `session_id` bilan qabul qilinadi; guruhda to'lov boshlash taqiqlanib, botning shaxsiy chatiga (`/start pay_ID`) xavfsiz yo'naltiriladi.
Rate Limiter: `BoundedCooldown` `time.monotonic()` va in-place xotira tozalash orqali to'liq xavfsizlandi.

3. AVTOMATLASHGAN TESTLAR VA VERIFIKATSIYA NATIJALARI

100% Yashil (21/21 Testlar)

Test Fayli / Modul	Sinovdan O'tgan Asosiy Funksiyalar	Natija
backend/test_audit_fixes.py	H1 No resurrection, H2 flag_is_true, H5 published check, D diagnostics, Monotonic limiter, E admin session, F group checkout redirect	✓ 7/7 PASSED (100%)
backend/test_ephemeral_video_and_upload.py	Ephemeral video payload, receiver_user_id, admin 1-click video saqlash, guruhga chiqarish, dars ruxsatlari	✓ 7/7 PASSED (100%)
backend/test_security_and_fixes.py	HMAC xavfsizlik, Admin ID tekshiruvi, Hamyon chegaralari, Tamper & Idempotency, Darslar modullari	✓ 6/6 PASSED (100%)
backend/test_bot_regression.py	Username tekshiruvi, xavfsiz chek caption, admin caption escape, hamyon qaytarish ledgeridan	✓ 5/5 PASSED (100%)
frontend (TypeScript & Vite)	Build (<code>tsc && vite build</code>) va Zod schema testlari (<code>vitest</code>)	✓ 0 Xatolik / 4/4 PASSED

```

1  """Course Academy Telegram Bot.
2
3  Aiogram ishlatmasdan Telegram Bot API + httpx asosida yozilgan: FastAPI bilan bitta
4  event loopda ishlaydi, ammo /start, katalog, checkout FSM, chek tasdiqlash, join
5  request himoyasi va admin buyruqlari to'liq ajratilgan handlerlarga ega.
6  """
7  from __future__ import annotations
8
9  import asyncio
10 import datetime as _dt
11 import html
12 import json
13 import logging
14 import secrets
15 import time
16 import uuid
17 from typing import Any, Dict, Optional, Tuple, List
18
19 import httpx
20
21 from app.api.ai import call_openrouter_api
22 from app.core.config import settings
23 from app.core.r2 import r2_client
24 from app.services.pricing import course_pricing
25 from app.services.purchases import (
26     approve_purchase,
27     is_join_request_authorized,
28     reject_purchase,
29     revoke_join_request_link,
30 )
31 from app.storage import get_store
32
33 logger = logging.getLogger(__name__)
34
35 API_URL = f"https://api.telegram.org/bot[REDACTED_TOKEN]""[REDACTED_BOT_TOKEN]"
36
37 # FSM faqat chek yuborishning qisqa oralig'i uchun kerak. Jarayon server qayta
38 # ishga tushsa toza holatga qaytadi; tasdiqlangan cheklar esa doim bazada qoladi.
39 _checkout_sessions: Dict[int, str] = {}
40
41 # Guruh AI yordamchisi uchun chat bo'yicha tezlik cheklovi (spam/kredit himoyasi)
42 _GROUP_AI_STATE: Dict[int, Dict[str, Any]] = {}
43 BOT_ID: Optional[int] = None
44
45 # Guruhda AI savol-javobni ishga tushiradigan sotuvga aloqador so'zlar
46 _GROUP_SALES_KEYWORDS = (
47     "kurs", "narx", "narxi", "chegirma", "to'lov", "tolov", "o'qimoq", "o'qimoqchi",
48     "o'rgan", "sertifikat", "platforma", "kreativ", "dizayn", "smm", "dasturlash",
49     "sun'iy intellekt", "prompt", "nechchi dars", "qancha turadi",
50 )
51
52 # Adminlar video yuklash holati: admin_id -> video_meta_dict
53 _pending_admin_videos: Dict[int, Dict[str, Any]] = {}
54
55
56 def flag_is_true(value: Any) -> bool:
57     """Qat'iy boolean tekshiruv (H2 yechimi: 'false', '0' kabi qiymatlarni rad etadi)."""
58     if value is True:
59         return True
60     if type(value) is int and value == 1:
61         return True
62     if isinstance(value, str) and value.strip().lower() in ("true", "1"):
63         return True
64     return False
65
66
67 def delivery_error_text(result: dict) -> str:
68     """Telegram xatoliklarini granulyar ajratib, talabaga aniq sababni ko'rsatish (D yechimi)."""
69     if result.get("delivery_unknown"):
70         return (
71             "Telegram javobi olinmadi. Video yetkazilgan bo'lishi mumkin; "
72             "avval chatni tekshiring."
73         )
74
75     code = result.get("error_code")
76     params = result.get("parameters")
77     params = params if isinstance(params, dict) else {}
78     desc = str(result.get("description") or "").lower()
79
80     if code == 429:
81         try:
82             delay = max(1, int(params.get("retry_after", 1)))
83             except (TypeError, ValueError):

```

```

84         delay = 1
85         return f"Telegram so'rovlar limiti. Kamida {delay} soniya kuting."
86
87     if code == 403:
88         return "Telegram amalni taqiqladi. Administratorga murojaat qiling."
89
90     if "query is too old" in desc or "query_id_invalid" in desc:
91         return "Tugma so'rovi eskirgan. Qaytadan bosing."
92
93     return "Video ochildi. Keyinroq urinib ko'ring yoki yordamga yozing."
94
95
96 class BoundedCooldown:
97     """Xotirasi cheklangan va eskirgan kalitlarni avtomatik tozalovchi rate limiter (monotonic va in-place)."""
98     def __init__(self, seconds: float = 5.0, max_keys: int = 50000):
99         self.seconds = seconds
100         self.max_keys = max_keys
101         self.items: Dict[Any, float] = {}
102
103     def take(self, key: Any) -> bool:
104         now = time.monotonic()
105         if len(self.items) >= self.max_keys:
106             cutoff = now - self.seconds
107             expired = [k for k, ts in self.items.items() if ts <= cutoff]
108             for k in expired:
109                 del self.items[k]
110             if len(self.items) >= self.max_keys:
111                 return False
112         last = self.items.get(key, 0.0)
113         if now - last < self.seconds:
114             return False
115         self.items[key] = now
116         return True
117
118     def clear(self) -> None:
119         self.items.clear()
120
121
122 # Studentlar uchun dars ko'rish rate limiteri
123 _lesson_cooldown = BoundedCooldown(seconds=5.0, max_keys=50000)
124 # Backwards-compatibility alias for tests
125 _user_lesson_rate_limits = _lesson_cooldown.items
126
127
128 def _uzs(amount: Any) -> str:
129     try:
130         return f"{int(amount or 0):,}".replace(",", " ") + " so'm"
131     except (ValueError, TypeError):
132         return "0 so'm"
133
134
135 def _escape(value: Any) -> str:
136     return html.escape(str(value or ""))
137
138
139 async def _telegram_call(
140     client: httpx.AsyncClient, method: str, payload: Dict[str, Any]
141 ) -> Dict[str, Any]:
142     """Telegram so'rovini bajaradi, rate limit (429) va xatolarni xavfsiz boshqaradi (M8 yechimi)."""
143     try:
144         response = await client.post(f"{API_URL}/{method}", json=payload, timeout=25.0)
145         try:
146             data = response.json()
147         except ValueError:
148             return {"ok": False, "error_code": response.status_code, "delivery_unknown": True}
149
150         if not isinstance(data, dict):
151             return {"ok": False, "error_code": response.status_code, "delivery_unknown": True}
152
153         if response.status_code == 200 and data.get("ok"):
154             return data
155
156         code = data.get("error_code") or response.status_code
157         params = data.get("parameters") or {}
158         if code == 429:
159             retry_after = params.get("retry_after", 1)
160             logger.warning("Telegram 429 Too Many Requests on %s: retry_after=%s", method, retry_after)
161         else:
162             logger.warning("Telegram %s xatosi: code=%s desc=%s", method, code, str(data.get("description") or "")[:200])
163
164         return {
165             "ok": False,
166             "error_code": code,
167             "description": data.get("description", ""),
168             "parameters": params,
169             "delivery_unknown": response.status_code >= 500,
170         }
171     except httpx.RequestError as exc:

```

```

172         logger.warning("Telegram %s tarmog'ida uzilish: %s", method, type(exc).__name__)
173         return {"ok": False, "error_code": 0, "delivery_unknown": True}
174     except Exception as exc:
175         logger.error("Telegram %s so'rovida kutilmagan xato: %s", method, exc)
176         return {"ok": False, "error_code": 0, "delivery_unknown": True}
177
178
179     async def send_tg_message(
180         client: httpx.AsyncClient,
181         chat_id: int,
182         text: str,
183         reply_markup: Optional[dict] = None,
184         protect_content: bool = True,
185     ) -> Dict[str, Any]:
186         payload: Dict[str, Any] = {
187             "chat_id": chat_id,
188             "text": text,
189             "parse_mode": "HTML",
190             "protect_content": protect_content,
191             "disable_web_page_preview": True,
192         }
193         if reply_markup:
194             payload["reply_markup"] = reply_markup
195         return await _telegram_call(client, "sendMessage", payload)
196
197
198     async def send_tg_photo(
199         client: httpx.AsyncClient,
200         chat_id: int,
201         photo: str,
202         caption: str,
203         reply_markup: Optional[dict] = None,
204     ) -> Dict[str, Any]:
205         payload: Dict[str, Any] = {
206             "chat_id": chat_id,
207             "photo": photo,
208             "caption": caption[:1024],
209             "parse_mode": "HTML",
210             "protect_content": True,
211         }
212         if reply_markup:
213             payload["reply_markup"] = reply_markup
214         return await _telegram_call(client, "sendPhoto", payload)
215
216
217     def is_admin(user_id: int | str) -> bool:
218         """str va int 64-bit xavfsiz admin tekshiruvi"""
219         if not user_id:
220             return False
221         try:
222             uid_int = int(str(user_id).strip())
223             if uid_int in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
224                 return True
225         except (ValueError, TypeError):
226             pass
227         uid_str = str(user_id).strip()
228         return uid_str in [str(a) for a in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]]
229
230
231     async def send_tg_video(
232         client: httpx.AsyncClient,
233         chat_id: int | str,
234         video: str,
235         caption: Optional[str] = None,
236         reply_markup: Optional[dict] = None,
237         protect_content: bool = True,
238         supports_streaming: bool = True,
239         duration: Optional[int] = None,
240         width: Optional[int] = None,
241         height: Optional[int] = None,
242     ) -> Dict[str, Any]:
243         payload: Dict[str, Any] = {
244             "chat_id": chat_id,
245             "video": video,
246             "protect_content": protect_content,
247             "supports_streaming": supports_streaming,
248         }
249         if caption:
250             payload["caption"] = caption[:1024]
251             payload["parse_mode"] = "HTML"
252         if reply_markup:
253             payload["reply_markup"] = reply_markup
254         if duration:
255             payload["duration"] = int(duration)
256         if width:
257             payload["width"] = int(width)
258         if height:
259             payload["height"] = int(height)

```

```

260     return await _telegram_call(client, "sendVideo", payload)
261
262
263 async def send_ephemeral_video(
264     client: httpx.AsyncClient,
265     chat_id: int | str,
266     video_file_id: str,
267     caption: str,
268     callback_user_id: int,
269     callback_query_id: str,
270     replace_callback_query_message: bool = False,
271     duration: Optional[int] = None,
272     width: Optional[int] = None,
273     height: Optional[int] = None,
274     protect_content: bool = True,
275 ) -> Dict[str, Any]:
276     """
277     Telegram Ephemeral Protected Video:
278     Telegram Bot API sendVideo chaqiruvini ephemeral_message_parameters bilan
279     faqat tugmani bosgan studentga va faqat o'sha clientga yuboriladi.
280     protect_content=True qat'iy qo'llaniladi.
281     """
282     payload: Dict[str, Any] = {
283         "chat_id": chat_id,
284         "video": video_file_id,
285         "caption": caption[:1024],
286         "parse_mode": "HTML",
287         "supports_streaming": True,
288         "protect_content": True,
289         "ephemeral_message_parameters": {
290             "receiver_user_id": int(callback_user_id),
291             "callback_query_id": str(callback_query_id),
292             "replace_callback_query_message": bool(replace_callback_query_message),
293         }
294     }
295     if duration:
296         payload["duration"] = int(duration)
297     if width:
298         payload["width"] = int(width)
299     if height:
300         payload["height"] = int(height)
301
302     return await _telegram_call(client, "sendVideo", payload)
303
304
305 async def send_tg_chat_action(client: httpx.AsyncClient, chat_id: int, action: str = "typing") -> None:
306     await _telegram_call(client, "sendChatAction", {"chat_id": chat_id, "action": action})
307
308
309 def _resolve_media_url(url: str) -> str:
310     """R2 media havolalarini Telegram yubora oladigan to'g'ridan-to'g'ri URL ga aylantiradi.
311
312     /api/media/{key} havolalari 307 redirect qaytaradi, Telegram sendPhoto esa
313     redirectlarni kuzatmaydi – shuning uchun kalit bo'yicha presigned URL ochamiz.
314     """
315     resolved = r2_client.resolve_stream_url(url, expires_in=86400)
316     return str(resolved or "").strip()
317
318
319 async def _answer_callback(
320     client: httpx.AsyncClient,
321     query_id: str,
322     text: str = "",
323     show_alert: bool = False,
324     url: Optional[str] = None,
325     cache_time: int = 0,
326 ) -> None:
327     payload: Dict[str, Any] = {"callback_query_id": query_id}
328     if text:
329         payload["text"] = text[:100]
330     if show_alert:
331         payload["show_alert"] = True
332     if url:
333         payload["url"] = url
334     if cache_time:
335         payload["cache_time"] = cache_time
336     await _telegram_call(client, "answerCallbackQuery", payload)
337
338
339 async def _ensure_user(tg_user: Dict[str, Any]) -> Optional[Dict[str, Any]]:
340     """Botdan foydalangan odamni users jadvalida yaratadi/yangi ma'lumotini yangilaydi."""
341     telegram_id = tg_user.get("id")
342     if not telegram_id:
343         return None
344
345     store = get_store()
346     name = " ".join(
347         part for part in (tg_user.get("first_name"), tg_user.get("last_name")) if part

```

```

348 ).strip() or "Talaba"
349 username = tg_user.get("username")
350 user = await store.get_user_by_tg(int(telegram_id))
351 if user:
352     updates = {
353         key: value
354         for key, value in {"name": name, "username": username}.items()
355         if value is not None and user.get(key) != value
356     }
357     if updates:
358         await store.update_user(user["id"], updates)
359         user.update(updates)
360     return user
361
362 role = "superadmin" if int(telegram_id) in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"] else "student"
363 try:
364     return await store.create_user(
365         {"telegram_id": int(telegram_id), "name": name, "username": username, "role": role}
366     )
367 except Exception as exc:
368     # Ikki update parallel kelgan bo'lsa, ikkinchisi mavjud userni qayta oladi.
369     logger.warning("Telegram user yaratilmadi: %s", exc)
370     return await store.get_user_by_tg(int(telegram_id))
371
372
373 def _course_keyboard(course: Dict[str, Any], index: int, total: int, is_group: bool = False) -> dict:
374     bot_user = settings.BOT_USERNAME.lstrip("@")
375     if is_group:
376         app_btn = {"text": "📄 Mini Appda ochish", "url": f"https://t.me/{bot_user}?start=catalog"}
377     else:
378         app_btn = {"text": "📄 Mini Appda batafsil", "web_app": {"url": f"{settings.WEBAPP_URL}#course_{course['id']}"}}
379     rows = [
380         [
381             {"text": "🔍 Oldingi", "callback_data": f"course:{(index - 1) % total}"},
382             {"text": f"{index + 1}/{total}", "callback_data": "noop"},
383             {"text": "Keyingi ➡", "callback_data": f"course:{(index + 1) % total}"},
384         ],
385         [{"text": "🇺🇦 To'lov qilish", "callback_data": f"pay:{course['id']}"},
386         [app_btn],
387     ]
388     return {"inline_keyboard": rows}
389
390
391 def _course_caption(course: Dict[str, Any], pricing: Optional[Dict[str, Any]] = None) -> str:
392     title = _escape(course.get("title") or "Kurs")
393     category = _escape(course.get("category") or "Premium ta'lim")
394     instructor = _escape(course.get("instructor_name") or "Kreativ AI ustozlari")
395
396     instructor_title = _escape(course.get("instructor_title") or "Ekspert")
397     # Faol "birinchi N kishi" chegirmasi bo'lsa – chegirmali yakuniy narx ko'rsatiladi
398     discount_active = bool((pricing or {}).get("discount_active"))
399     final_price = (pricing or {}).get("final_price")
400     shown_price = final_price if discount_active and final_price is not None else course.get("price")
401     shown_old = course.get("price") if discount_active and final_price is not None else course.get("old_price")
402     old_price_line = f"\n<s>{_uzs(shown_old)}</s>" if shown_old and shown_old > (shown_price or 0) else ""
403     discount = course.get("discount_percent")
404     if discount_active and discount:
405         spots = (pricing or {}).get("discount_spots_left")
406         discount_line = f"\n🔥 <b>–{int(discount)}% chegirma</b> – birinchi {int(course.get('discount_limit') or 0)} kishi uchun"
407         if spots is not None:
408             discount_line += f", {int(spots)} ta joy qoldi"
409     else:
410         discount_line = f" 🔥 -{discount}%" if discount and not course.get("discount_limit") else ""
411     description = _escape(course.get("short_description") or course.get("description") or "")
412     if len(description) > 290:
413         description = description[:287].rstrip() + "..."
414     return (
415         f"📌 <b>{title}</b>\n"
416         f"📂 {category}\n"
417         f"📅 {int(course.get('lesson_count') or 0)} ta dars · ⌚ {_escape(course.get('duration') or 'Davomiyligi ko'rsatiladi')}\n"
418         f"💰 <b>{_uzs(shown_price)}</b>{old_price_line}{discount_line}\n"
419         f"👤 <b>{instructor}</b> – {instructor_title}\n"
420         f"⭐ {course.get('rating') or 5.0}/5\n"
421         f"{description}"
422     )
423
424
425 async def show_course_card(client: httpx.AsyncClient, chat_id: int, index: int) -> None:
426     store = get_store()
427     courses = await store.list_courses(published_only=True)
428     if not courses:
429         await send_tg_message(client, chat_id, "🇺🇦 Hozircha kurslar tayyorlanmoqda. Tez orada qayta ko'ring!")
430         return
431
432     index %= len(courses)
433     course = courses[index]
434     try:
435         pricing = await course_pricing(store, course)

```

```

436 except Exception:
437     pricing = {"discount_active": False, "discount_spots_left": None, "final_price": course.get("price")}
438 is_group = int(chat_id) < 0
439 keyboard = _course_keyboard(course, index, len(courses), is_group=is_group)
440 caption = _course_caption(course, pricing)
441 cover_url = _resolve_media_url(course.get("cover_url") or "")
442 if cover_url.startswith(("https://", "http://")):
443     result = await send_tg_photo(client, chat_id, cover_url, caption, keyboard)
444     if result.get("ok"):
445         return
446 await send_tg_message(client, chat_id, caption, keyboard)
447
448
449 async def _get_active_card_info() -> Dict[str, str]:
450     store = get_store()
451     card_num = settings.CARD_NUMBER
452     card_holder = settings.CARD HOLDER
453     bank_name = settings.CARD_BANK
454     try:
455         saved_pay = await store.get_setting("payment_settings")
456         if saved_pay:
457             data = json.loads(saved_pay)
458             if data.get("card_number"):
459                 card_num = data["card_number"]
460             if data.get("card_holder"):
461                 card_holder = data["card_holder"]
462             if data.get("bank_name"):
463                 bank_name = data["bank_name"]
464     except Exception:
465         pass
466     return {"card_number": card_num, "card_holder": card_holder, "bank_name": bank_name}
467
468
469 async def _send_payment_info(client: httpx.AsyncClient, chat_id: int) -> None:
470     card = await _get_active_card_info()
471     text = (
472         "📌 <b>Kreativ AI – To'lov rekvizitlari</b>\n\n"
473         f"🏦 Bank: <b>{_escape(card['bank_name'])}</b>\n\n"
474         f"📄 Karta raqami (nuxtalash uchun bosing):\n\n"
475         f"<code>{_escape(card['card_number'])}</code>\n\n"
476         f"👤 Qabul qiluvchi: <b>{_escape(card['card_holder'])}</b>\n\n"
477         "💡 <b>To'lov tartibi:</b>\n\n"
478         "1. Karta raqamidan nusxa oling va ilovangizdan pul o'tkazing.\n\n"
479         "2. To'lov chekini skrinshot qilib, shu chatga rasm ko'rinishida yuboring.\n\n"
480         "3. Admin tasdiqlashi bilan darslar va yopiq kanal avtomatik ochiladi."
481     )
482     rows = [
483         [{"text": "📌 Kreativ AI – Mini App", "web_app": {"url": settings.WEBAPP_URL}},
484         [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}],
485     ]
486     await send_tg_message(
487         client,
488         chat_id,
489         text,
490         {"inline_keyboard": rows},
491     )
492
493
494 async def _start_checkout(client: httpx.AsyncClient, chat_id: int, user: Dict[str, Any], course_id: str) -> None:
495     # Guruhda to'lov boshlanmasligi kerak – maxfiylik va xavfsizlik uchun faqat shaxsiy chatda (F yechimi)
496     user_tg_id = int(user.get("telegram_id") or 0)
497     if int(chat_id) != user_tg_id:
498         bot_user = (settings.BOT_USERNAME or "").lstrip("@")
499         await send_tg_message(
500             client,
501             chat_id,
502             "⚠️ Xavfsizlik va maxfiylik maqsadida to'lov faqat botning shaxsiy chatida amalga oshiriladi.",
503             {
504                 "inline_keyboard": [[{
505                     "text": "📌 Botda to'lovni boshlash",
506                     "url": f"https://t.me/{bot_user}?start=pay_{course_id}" if bot_user else settings.WEBAPP_URL,
507                 }]]
508             },
509         )
510         return
511
512     store = get_store()
513     course = await store.get_course(course_id)
514     if not course or not flag_is_true(course.get("published", True)):
515         await send_tg_message(client, chat_id, "⚠️ Bu kurs hozir mavjud emas.")
516         return
517     card = await _get_active_card_info()
518     # Chegirma faol bo'lsa bot ham mini-app bilan bir xil yakuniy narxni ko'rsatadi
519     try:
520         pricing = await course_pricing(store, course)
521     except Exception:
522         pricing = {"discount_active": False, "discount_spots_left": None, "final_price": course.get("price")}
523     amount = pricing["final_price"]

```

```

524 price_block = f"💰 Summa: <b>{_uzs(amount)}</b>"
525 if pricing["discount_active"] and course.get("discount_percent"):
526     price_block = (
527         f"💰 Summa: <s>{_uzs(course.get('price'))}</s> → <b>{_uzs(amount)}</b>\n"
528         f"🔥 <b>-{int(course['discount_percent'])}% chegirma</b> – birinchi {int(course.get('discount_limit') or 0)} kishi uchun"
529         + f"({f", {int(pricing['discount_spots_left'])} ta joy qoldi" if pricing.get("discount_spots_left") is not None else ""})"
530     )
531 # Hamyon balansi: bot oqimida hamyondan yechish yo'q (bu faqat Mini App checkoutda),
532 # shuning uchun balans bor foydalanuvchiga aniq yo'l-yo'rig' ko'rsatamiz.
533 wallet_line = ""
534 try:
535     from app.services import wallet as wallet_service
536     wallet = await wallet_service.get_wallet(store, str(user.get("id") or ""))
537     if wallet.get("balance", 0) > 0:
538         wallet_line = (
539             f"\n💰 <b>Hamyon balansingiz: {_uzs(wallet['balance'])}</b>\n"
540             "Chekni shu chatga yuborsangiz butun summa kartadan hisoblanadi. Hamyondan "
541             f"aniq summa ajratib, qolganini kartadan to'lash uchun xaridni <b>Mini Appda</b> davom ettiring."
542         )
543 except Exception:
544     logger.exception("Hamyon balansini o'qishda xato (user=%s)", user.get("id"))
545 _checkout_sessions[int(user["telegram_id"])] = course["id"]
546 await send_tg_message(
547     client,
548     chat_id,
549     f"👋 <b>To'lovni amalga oshirish</b>\n\n"
550     f"📄 Kurs: <b>{_escape(course['title'])}</b>\n"
551     f"💰 {price_block}{wallet_line}\n\n"
552     f"🏦 Bank: <b>{_escape(card['bank_name'])}</b>\n"
553     f"📄 Karta raqami (nusxalash uchun bosing):\n"
554     f"<code>{_escape(card['card_number'])}</code>\n"
555     f"👤 Qabul qiluvchi: <b>{_escape(card['card_holder'])}</b>\n\n"
556     "To'lovni amalga oshiring va <b>chek skrinshotini rasm ko'rinishida shu chatga yuboring</b>.\n"
557     "Admin tekshirishi bilan darslar sizga ochiladi.",
558     {"inline_keyboard": [
559         [{"text": "📄 Mini Appda ko'rish", "web_app": {"url": f"{settings.WEBAPP_URL}#course_{course['id']}}"}],
560         [{"text": "❌ Bekor qilish", "callback_data": "checkout:cancel"}]
561     ]},
562 )
563
564
565
566 async def _notify_admins_about_receipt(
567     client: httpx.AsyncClient, purchase: Dict[str, Any], photo_file_id: str
568 ) -> int:
569     """Yangi chekni barcha superadminlarga yuboradi."""
570     username = str(purchase.get("username") or "").rstrip("@")
571     username_label = f"@{_escape(username)}" if username else "username yo'q"
572     caption = (
573         f"📄 <b>YANGI TO'LOV CHEKI KELD!</b>\n\n"
574         f"👤 <b>Talaba:</b> {_escape(purchase.get('student_name') or 'Talaba')} ({username_label})\n"
575         f"📄 <b>Telegram ID:</b> <code>{purchase.get('telegram_id') or 0}</code>\n"
576         f"📄 <b>Kurs:</b> {_escape(purchase.get('course_title') or 'Kurs')}\n"
577         f"💰 <b>Summa:</b> {_uzs(purchase.get('amount'))}\n"
578         f"📄 <b>To'lov turi:</b> {_escape(purchase.get('payment_method') or 'card').upper()}\n"
579         f"📄 <b>Buyurtma ID:</b> <code>{_escape(purchase.get('transaction_id'))}</code>\n\n"
580         "Chekni tekshirib, tugma orqali qaror qabul qiling."
581     )
582     keyboard = {
583         "inline_keyboard": [
584             [
585                 {"text": "✅ Tasdiqlash", "callback_data": f"approve_{purchase['transaction_id']}"},
586                 {"text": "❌ Rad etish", "callback_data": f"reject_{purchase['transaction_id']}"},
587             ],
588             [{"text": "📄 Admin panel", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}],
589         ]
590     }
591     notified = 0
592     for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
593         result = await send_tg_photo(client, admin_id, photo_file_id, caption, keyboard)
594         if result.get("ok"):
595             notified += 1
596     return notified
597
598
599 MAX_RECEIPT_BYTES = 8 * 1024 * 1024
600
601
602 async def _store_telegram_receipt(
603     client: httpx.AsyncClient, file_id: str, transaction_id: str
604 ) -> Optional[str]:
605     """Telegramdan kelgan chekni oqimli (streaming) yuklab, R2 ga ko'chiradi (M9 yechimi)."""
606     try:
607         file_data = await _telegram_call(client, "getFile", {"file_id": file_id})
608         file_path = (file_data.get("result") or {}).get("file_path")
609         if not file_path:
610             return None

```

```

612 url = f"https://api.telegram.org/file/bot{"[REDACTED_BOT_TOKEN]"}/{file_path}"
613 async with client.stream("GET", url, timeout=30.0, follow_redirects=False) as response:
614     if response.status_code != 200:
615         return None
616     raw_len = response.headers.get("content-length")
617     if raw_len:
618         try:
619             if int(raw_len) > MAX_RECEIPT_BYTES:
620                 logger.warning("Chek hajmi limitdan katta: %s bayt", raw_len)
621                 return None
622             except ValueError:
623                 pass
624
625     body = bytearray()
626     async for chunk in response.aiter_bytes(chunk_size=64 * 1024):
627         if len(body) + len(chunk) > MAX_RECEIPT_BYTES:
628             logger.warning("Chek oqimi 8MB limitdan oshib ketdi, to'xtatildi")
629             return None
630         body.extend(chunk)
631
632     if not body:
633         return None
634     image_bytes = bytes(body)
635
636     ext = str(file_path).rsplit(".", 1)[-1].lower() if "." in str(file_path) else "jpg"
637     if ext not in {"jpg", "jpeg", "png", "webp"}:
638         ext = "jpg"
639     content_type = "image/jpeg" if ext in {"jpg", "jpeg"} else f"image/{ext}"
640     return await asyncio.to_thread(
641         r2_client.upload_bytes, f"receipts/{transaction_id}.{ext}", image_bytes, content_type
642     ) or None
643 except Exception as exc:
644     logger.warning("Telegram cheki yuklanmadi/R2 ga ko'chirilmadi: %s", exc)
645     return None
646
647
648 async def _handle_receipt_photo(client: httpx.AsyncClient, message: Dict[str, Any], user: Dict[str, Any]) -> bool:
649     telegram_id = int(user.get("telegram_id") or 0)
650     course_id = _checkout_sessions.get(telegram_id)
651     photos = message.get("photo") or []
652     if not course_id or not photos:
653         return False
654
655     store = get_store()
656     course = await store.get_course(course_id)
657     if not course:
658         _checkout_sessions.pop(telegram_id, None)
659         await send_tg_message(client, telegram_id, " ⚠️ Kurs topilmadi. Iltimos, katalogdan qayta tanlang.")
660         return True
661
662     file_id = photos[-1].get("file_id")
663     if not file_id:
664         await send_tg_message(client, telegram_id, " ⚠️ Rasmni o'qib bo'lmadi. Iltimos, yana yuboring.")
665         return True
666
667     # Foydalanuvchi captioni: uzunligini cheklab, tizim teglarini (SYSTEM[...],
668     # 'wallet:', 'promo:') soxtalashtirib bo'lmas qilamiz – rad etilganda hamyon
669     # qaytarish oqimi shu commentga qaram bo'lgani uchun uni tozalash muhim.
670     raw_caption = str(message.get("caption") or "").strip()[:300]
671     clean_caption = (
672         raw_caption.replace("SYSTEM[", "SYSTEM[").replace("wallet:", "wallet_").replace("promo:", "promo_")
673     ) or None
674
675
676     transaction_id = f"rcp_{uuid.uuid4().hex[:12]}"
677     receipt_url = await _store_telegram_receipt(client, file_id, transaction_id)
678     # Botdan kelgan chek ham mini-app bilan bir xil chegirmali narxda qayd etiladi
679     try:
680         pricing = await course_pricing(store, course)
681     except Exception:
682         pricing = {"final_price": course.get("price")}
683     purchase = await store.create_purchase(
684         {
685             "user_id": user["id"],
686             "course_id": course["id"],
687             "course_title": course["title"],
688             "amount": pricing["final_price"],
689             "status": "pending_approval",
690             "payment_method": "telegram_receipt",
691             "transaction_id": transaction_id,
692             "telegram_id": telegram_id,
693             "student_name": user.get("name") or "Talaba",
694             "username": user.get("username"),
695             # Telegram file_id boshqa chatga sendPhoto qilish uchun yetarli; chekning o'zi Telegramda qoladi.
696             "receipt_image_url": receipt_url or f"tg-file:{file_id}",
697             "comment": clean_caption,
698         }
699     )

```

```

700 _checkout_sessions.pop(telegram_id, None)
701 notified = await _notify_admins_about_receipt(client, purchase, file_id)
702 await send_tg_message(
703     client,
704     telegram_id,
705     f"✅ <b>Chekingiz qabul qilindi!</b>\n\n"
706     f"<b>{_escape(course['title'])}</b> uchun to'lovingiz tekshiruvga yuborildi. "
707     f"Admin tasdiqlashi bilan kurs va yopiq kanal havolasi avtomatik yuboriladi.",
708 )
709 if not notified:
710     logger.warning("Chek adminlarga yuborilmadi: %s", purchase["transaction_id"])
711 return True
712
713
714 async def _get_effective_target_group(store, course_id: Optional[str] = None) -> Tuple[Optional[str], str]:
715     """Hozirda faol bo'lgan Target Guruh ID va nomini olish."""
716     if course_id:
717         course = await store.get_course(course_id)
718         if course and course.get("telegram_channel_id"):
719             return str(course["telegram_channel_id"]), str(course.get("title") or "Kurs Guruhi")
720
721     dyn_id = await store.get_setting("target_group_id")
722     dyn_title = await store.get_setting("target_group_title") or "Kurs Guruhi"
723     if dyn_id:
724         return str(dyn_id), str(dyn_title)
725
726     cfg_id = getattr(settings, "TARGET_GROUP_ID", "")
727     if cfg_id:
728         return str(cfg_id), "Target Guruh"
729
730     return None, ""
731
732
733 async def publish_lesson_to_group(
734     client: httpx.AsyncClient,
735     lesson_id: str | int,
736     admin_chat_id: int,
737 ) -> bool:
738     """Darsni Target Supergroupga faqat inline play tugmasi bilan chiqarish."""
739     store = get_store()
740     lesson = await store.get_lesson(lesson_id)
741     if not lesson:
742         courses = await store.list_courses(published_only=False)
743         for c in courses:
744             from app.api.courses import resolve_course_modules
745             for m in resolve_course_modules(c):
746                 for l in m.get("lessons", []):
747                     if str(l.get("id")) == str(lesson_id):
748                         lesson = {"**1", "course_id": c["id"]}
749                         break
750             if lesson:
751                 break
752         if lesson:
753             break
754
755     if not lesson:
756         await send_tg_message(client, admin_chat_id, f"❌ #{lesson_id} dars topilmadi.")
757         return False
758
759     if lesson.get("published_message_id") and lesson.get("published"):
760         await send_tg_message(
761             client,
762             admin_chat_id,
763             f"📄 <b>{_escape(lesson.get('title') or 'Dars')}</b> allaqachon guruhga chiqarilgan (Xabar ID: <code>{lesson.get('published_message_id')}</code>).",
764         )
765         return True
766
767     target_id, target_title = await _get_effective_target_group(store, lesson.get("course_id"))
768     if not target_id:
769         await send_tg_message(
770             client,
771             admin_chat_id,
772             f"❌ <b>Xatolik:</b> Darslar chiqariladigan Target Guruh hali belgilanmagan!\n\n"
773             f"Botni kurs guruhingizga admin qilib qo'shing va o'sha guruhda <code>set_group</code> buyrug'ini yuboring.",
774         )
775         return False
776
777     lesson_title = lesson.get("title") or "Dars"
778     post_text = (
779         f"🎥 <b>{_escape(lesson_title)}</b>\n\n"
780         f"Videon ko'rish uchun pastdagi tugmani bosing."
781     )
782     inline_keyboard = {
783         "inline_keyboard": [
784             [
785                 {
786                     "text": f"📄 {_escape(lesson_title[:28])}ni ko'rish",
787                     "callback_data": f"lesson:{lesson_id}",
788                 }
789             ]
790         ]
791     }

```

```

788         }
789     ]
790 ]
791 }
792
793 tg_target = int(target_id) if str(target_id).lstrip("-").isdigit() else target_id
794 res = await send_tg_message(
795     client,
796     tg_target,
797     post_text,
798     reply_markup=inline_keyboard,
799     protect_content=getattr(settings, "PROTECT_CONTENT", True),
800 )
801 if res.get("ok"):
802     msg_id = (res.get("result") or {}).get("message_id")
803     try:
804         await store.update_lesson(lesson_id, {"published_message_id": msg_id, "published": True})
805     except Exception:
806         pass
807     await send_tg_message(
808         client,
809         admin_chat_id,
810         f"✅ <b>{_escape(lesson_title)}</b> darsi <b>{_escape(target_title)}</b> guruhiga chiqarildi!\n"
811         f"Xabar ID: <code>{msg_id}</code>",
812     )
813     return True
814 else:
815     err = res.get("description") or "Noma'lum xatolik"
816     await send_tg_message(
817         client,
818         admin_chat_id,
819         f"❌ Guruhga chiqarishda xatolik yuz berdi: <i>{_escape(err)}</i>\n\n"
820         "Bot guruhda administrator ekanligini va xabar yozish huquqi borligini tekshiring.",
821     )
822     return False
823
824
825 class LessonAccessDenied(Exception):
826     def __init__(self, message: str, can_buy: bool = False, course_id: Optional[str] = None):
827         super().__init__(message)
828         self.message = message
829         self.can_buy = can_buy
830         self.course_id = course_id
831
832
833 async def require_lesson_access(
834     user_id: int,
835     lesson_id: str | int,
836 ) -> Tuple[Any, Dict[str, Any], Optional[Dict[str, Any]]]:
837     """
838     Yagona va qat'iy (fail-closed) darsga kirish huquqi tekshiruvi (H1 & H2 yechimi):
839     - user_id musbat son bo'lishi shart
840     - Global va dars blacklist tekshiruvi (fail-closed)
841     - Dars va Kurs mavjudligi hamda published holati tekshiruvi
842     - Preview yoki Admin bo'lsa ruxsat
843     - Boshqa holatda aktiv (active) enrollment mavjudligi tekshiriladi
844     """
845     if not isinstance(user_id, int) or user_id <= 0:
846         raise LessonAccessDenied("Foydalanuvchi aniqlanmadi.")
847
848     store = get_store()
849
850     # 1. Darsni bazadan qidirish
851     lesson = await store.get_lesson(lesson_id)
852     target_course_id = (lesson or {}).get("course_id")
853     if not lesson:
854         courses = await store.list_courses(published_only=False)
855         for c in courses:
856             from app.api.courses import resolve_course_modules
857             for m in resolve_course_modules(c):
858                 for l in m.get("lessons", []):
859                     if str(l.get("id")) == str(lesson_id):
860                         lesson = {"**1", "course_id": c["id"]}
861                         target_course_id = c["id"]
862                         break
863             if lesson:
864                 break
865         if lesson:
866             break
867
868     if not lesson:
869         raise LessonAccessDenied("Dars topilmadi yoki o'chirilgan.")
870
871     # 2. Admin huquqi
872     if is_admin(user_id):
873         return store, lesson, await store.get_user_by_tg(user_id)
874
875     # 3. Global blacklist (Fail-closed)

```

```

876     try:
877         if await store.is_user_blocked(user_id):
878             raise LessonAccessDenied("Hisobingiz bloklangan.")
879     except LessonAccessDenied:
880         raise
881     except Exception as exc:
882         logger.error("Blocklist tekshiruvida xato: %s", exc)
883         raise LessonAccessDenied("Xavfsizlik tekshiruvida xatolik yuz berdi. Keyinroq urinib ko'ring.")
884
885     # 4. Dars blocklisti (Fail-closed)
886     try:
887         is_blocked, reason = await store.is_user_blocked_for_lesson(user_id, lesson_id)
888         if is_blocked:
889             msg = "Ushbu darsdan foydalanish ma'muriyat tomonidan cheklangan."
890             if reason:
891                 msg += f"\nSabab: {reason}"
892             raise LessonAccessDenied(msg)
893     except LessonAccessDenied:
894         raise
895     except Exception as exc:
896         logger.error("Dars blocklist tekshiruvida xato: %s", exc)
897         raise LessonAccessDenied("Xavfsizlik tekshiruvida xatolik yuz berdi. Keyinroq urinib ko'ring.")
898
899     # 5. Kurs tekshiruvi: Dars birlashtirilgan kurs bo'lishi shart (H2 yechimi)
900     # 5. Kurs va darsning published holati tekshiruvi (H2 & H5 yechimi)
901     if not target_course_id:
902         raise LessonAccessDenied("Ushbu darsga kirish hozircha mumkin emas (kurs belgilanmagan).")
903
904     course = await store.get_course(target_course_id)
905     if not course or not flag_is_true(course.get("published", True)):
906         raise LessonAccessDenied("Kurs hali ochilmagan yoki nofaol.")
907
908     if lesson.get("published") is not None and not flag_is_true(lesson.get("published")):
909         raise LessonAccessDenied("Dars hali ochilmagan.")
910
911     # 6. Preview dars bo'lsa darhol ruxsat (Qat'iy boolean tekshiruv)
912     if flag_is_true(lesson.get("is_preview")):
913         return store, lesson, await store.get_user_by_tg(user_id)
914
915     # 7. Foydalanuvchi va Faol Enrollment tekshiruvi (H1 yechimi: eski xarid orqali resurrection olib tashlandi)
916     user = await store.get_user_by_tg(user_id)
917     if not user:
918         raise LessonAccessDenied("Avval botimizga kiring: /start", can_buy=True, course_id=target_course_id)
919
920     enr = await store.get_enrollment(user["id"], target_course_id)
921     if not enr or enr.get("status") != "active":
922         raise LessonAccessDenied(
923             f"❗ «{lesson.get('title') or 'Dars'}» himoyalangan dars.\n\nUshbu kurs uchun faol ruxsat mavjud emas.",
924             can_buy=enr is None,
925             course_id=target_course_id,
926         )
927
928     return store, lesson, user
929
930
931 async def deliver_private_lesson(
932     client: httpx.AsyncClient,
933     user_id: int,
934     lesson_id: str | int,
935     query_id: Optional[str] = None,
936 ) -> bool:
937     """
938     Shaxsiy chatga dars yo'naltirish:
939     Oddiy talabaga pullik dars xom video fayl sifatida yuborilmaydi (AyuGram himoyasi).
940     Buning o'rniga yopiq o'quv guruhida ephemeral ko'rish yoki Mini App bo'yicha yo'riqnoma beriladi.
941     Faqat adminlar yoki bepul preview darslar uchun to'g'ridan-to'g'ri yuboriladi.
942     """
943     store = get_store()
944     try:
945         store, lesson, user = await require_lesson_access(user_id, lesson_id)
946     except LessonAccessDenied as exc:
947         if query_id:
948             bot_user = settings.BOT_USERNAME.lstrip("@")
949             if exc.can_buy and exc.course_id:
950                 await _answer_callback(
951                     client,
952                     query_id,
953                     exc.message[:200],
954                     show_alert=True,
955                     url=f"https://t.me/{bot_user}?start=catalog",
956                 )
957             else:
958                 await _answer_callback(client, query_id, exc.message[:200], show_alert=True)
959         else:
960             await send_tg_message(client, user_id, f"⚠️ {_escape(exc.message)}")
961         return False
962     except Exception as exc:
963         logger.error("Lesson auth unexpected error: %s", exc)

```

```

964         if query_id:
965             await _answer_callback(client, query_id, " ⚠️ Ruqsatni tekshirishda xatolik.", show_alert=True)
966         else:
967             await send_tg_message(client, user_id, " ⚠️ Ruqsatni tekshirishda xatolik yuz berdi.")
968         return False
969
970     video_file_id = lesson.get("video_file_id") or lesson.get("telegram_file_id") or lesson.get("file_id")
971     if not video_file_id and str(lesson.get("video_url") or "").startswith("tg-file:"):
972         video_file_id = str(lesson["video_url"]).split("tg-file:", 1)[1]
973
974     if not video_file_id:
975         if query_id:
976             await _answer_callback(client, query_id, " ❌ Video fayli topilmadi.", show_alert=True)
977         else:
978             await send_tg_message(client, user_id, " ❌ Video fayli topilmadi.")
979         return False
980
981     raw_title = str(lesson.get("title") or "Dars")[:180]
982     caption = f" 📌 <b>{_escape(raw_title)}</b>\n\n 📺 <i>Himoyalangan dars videosi</i>"
983     duration = lesson.get("duration_seconds")
984     if duration is None and isinstance(lesson.get("duration"), int):
985         duration = lesson["duration"]
986
987     # Admin yoki preview dars bo'lmasa, shaxsiy chatga xom video fayl yuborilmaydi
988     # Qat'iy boolean tekshiruv (H2 yechimi)
989     if not is_admin(user_id) and not flag_is_true(lesson.get("is_preview")):
990         target_course_id = lesson.get("course_id")
991         target_id, target_title = await _get_effective_target_group(store, target_course_id)
992         group_url = None
993         if target_title and str(target_title).startswith("@"):
994             group_url = f"https://t.me/{str(target_title).lstrip('@')}"
995         elif target_id:
996             group_url = f"https://t.me/c/{str(target_id).replace('-100', '')}"
997
998     keyboard_rows = [
999         [{"text": " 📖 Kurs ma'lumotlari (Mini App)", "web_app": {"url": f"{settings.WEBAPP_URL}#course_{target_course_id}"}}],
1000     ]
1001     if group_url:
1002         keyboard_rows.insert(0, [{"text": " 🗺️ O'quv guruhiga o'tish", "url": group_url}])
1003
1004     guide_text = (
1005         f" 📌 <b>{_escape(raw_title)}</b>\n\n"
1006         f" 📺 <b>Himoyalangan dars videosi</b>\n\n"
1007         f"Barcha to'liq video darslar faqat rasmiy yopiq <b>O'quv guruhida</b> namoyish etiladi va shaxsiy chatga xom fayl ko'rinishida yuborilmaydi.\n\n"
1008         f"Darsni tomosha qilish uchun o'quv guruhidagi ushbu dars e'lonidagi <b>📺 Darsni ko'rish</b> tugmasini bosib – video faqat sizning ekraningizda och
1009     iladi!"
1010 )
1011     if query_id:
1012         await _answer_callback(client, query_id)
1013         await send_tg_message(client, user_id, guide_text, reply_markup={"inline_keyboard": keyboard_rows})
1014         return True
1015
1016     # Faqat Admin yoki Preview darslar uchun shaxsiy chatga jo'natish
1017     res = await send_tg_video(
1018         client=client,
1019         chat_id=user_id,
1020         video=video_file_id,
1021         caption=caption,
1022         protect_content=getattr(settings, "PROTECT_CONTENT", True),
1023         supports_streaming=True,
1024         duration=duration,
1025         width=lesson.get("width"),
1026         height=lesson.get("height"),
1027     )
1028
1029     if not res.get("ok"):
1030         err_code = res.get("error_code")
1031         desc = str(res.get("description") or "")
1032         logger.warning("Private video delivery failed to %s: code=%s desc=%s", user_id, err_code, desc)
1033         if query_id:
1034             bot_user = settings.BOT_USERNAME.lstrip("@")
1035             if err_code == 403 or "blocked" in desc.lower() or "chat not found" in desc.lower():
1036                 await _answer_callback(
1037                     client,
1038                     query_id,
1039                     " ⚠️ Videoni olish uchun avval botimizga kiring!",
1040                     show_alert=True,
1041                     url=f"https://t.me/{bot_user}?start=lesson_{lesson_id}",
1042                 )
1043             else:
1044                 await _answer_callback(client, query_id, " ⚠️ Video yuborishda xatolik yuz berdi.", show_alert=True)
1045         else:
1046             await send_tg_message(client, user_id, " ❌ Videoni yuborishda xatolik yuz berdi.")
1047         return False
1048
1049     # Telemetriya va log
1050     try:
1051         await store.log_access({

```

```

1051         "lesson_id": str(lesson["id"]),
1052         "user_id": user_id,
1053         "mode": "private_protected",
1054     })
1055     target_course_id = lesson.get("course_id")
1056     if user and target_course_id:
1057         await store.upsert_progress(
1058             user["id"], target_course_id, str(lesson["id"]), completed=False
1059         )
1060     except Exception as exc:
1061         logger.error("Access log update error: %s", exc)
1062
1063     if query_id:
1064         await _answer_callback(client, query_id, "✅ Dars videosi shaxsiy botingizga yuborildi!")
1065     return True
1066
1067
1068 async def handle_student_ephemeral_lesson(
1069     client: httpx.AsyncClient,
1070     callback_query: Dict[str, Any],
1071     lesson_id: str | int,
1072 ) -> None:
1073     """Student guruhda dars tugmasini bosganda Ephemeral video (faqat o'sha talabaga ko'rinadigan) yetkazish."""
1074     query_id = callback_query.get("id", "")
1075     tg_user = callback_query.get("from") or {}
1076     user_id = int(tg_user.get("id") or 0)
1077     chat = (callback_query.get("message") or {}).get("chat") or {}
1078     chat_id = chat.get("id")
1079     chat_type = str(chat.get("type") or "")
1080
1081     # 1. Rate limiting (BoundedCooldown orqali xotira himoyalangan)
1082     if not _lesson_cooldown.take(user_id):
1083         await _answer_callback(client, query_id, "Iltimos, biroz kuting (5 soniya).")
1084         return
1085
1086     # 2. Xavfsiz avtorizatsiya tekshiruvi (Mini App xaridi / faol enrollment)
1087     store = get_store()
1088     try:
1089         store, lesson, user = await require_lesson_access(user_id, lesson_id)
1090     except LessonAccessDenied as exc:
1091         bot_user = settings.BOT_USERNAME.lstrip("@")
1092         if exc.can_buy and exc.course_id:
1093             await _answer_callback(
1094                 client,
1095                 query_id,
1096                 exc.message[:200],
1097                 show_alert=True,
1098                 url=f"https://t.me/{bot_user}?start=catalog",
1099             )
1100         else:
1101             await _answer_callback(client, query_id, exc.message[:200], show_alert=True)
1102         return
1103     except Exception as exc:
1104         logger.error("Lesson auth unexpected error: %s", exc)
1105         await _answer_callback(client, query_id, "⚠️ Ruqsatni tekshirishda xatolik.", show_alert=True)
1106         return
1107
1108     video_file_id = lesson.get("video_file_id") or lesson.get("telegram_file_id") or lesson.get("file_id")
1109     if not video_file_id and str(lesson.get("video_url") or "").startswith("tg-file:"):
1110         video_file_id = str(lesson["video_url"]).split("tg-file:", 1)[1]
1111
1112     if not video_file_id:
1113         await _answer_callback(client, query_id, "❌ Video fayli topilmadi.", show_alert=True)
1114         return
1115
1116     raw_title = str(lesson.get("title") or "Dars")[:180]
1117     caption = f"📌 <b>{<_escape(raw_title)</b></b>\n\n📅 <i>Himoyalangan dars videosi</i>"
1118     duration = lesson.get("duration_seconds")
1119     if duration is None and isinstance(lesson.get("duration"), int):
1120         duration = lesson["duration"]
1121
1122     # 3. Guruhda bosilgan bo'lsa -> Faqat Ephemeral Video (AyuGram yoki mod klientlar uchun privat fallback qilinmaydi!)
1123     if chat_id and chat_type in ("group", "supergroup"):
1124         res = await send_ephemeral_video(
1125             client=client,
1126             chat_id=chat_id,
1127             video_file_id=video_file_id,
1128             caption=caption,
1129             callback_user_id=user_id,
1130             callback_query_id=query_id,
1131             replace_callback_query_message=False,
1132             duration=duration,
1133             width=lesson.get("width"),
1134             height=lesson.get("height"),
1135             protect_content=True,
1136         )
1137         if res.get("ok"):
1138             await _answer_callback(client, query_id, "✅ Dars videosi faqat siz uchun ochildi!")

```

```

1139         try:
1140             await store.log_access({
1141                 "lesson_id": str(lesson["id"]),
1142                 "user_id": user_id,
1143                 "mode": "group_ephemeral",
1144             })
1145             target_course_id = lesson.get("course_id")
1146             if user and target_course_id:
1147                 await store.upsert_progress(
1148                     user["id"], target_course_id, str(lesson["id"]), completed=False
1149                 )
1150             except Exception as exc:
1151                 logger.error("Access log update error: %s", exc)
1152         else:
1153             # Ephemeral yetkazilmasa, privat fallback qilinmaydi (Anti-AyuGram)
1154             logger.warning("Ephemeral delivery failed in group for %s: %s", user_id, res)
1155             await _answer_callback(
1156                 client,
1157                 query_id,
1158                 delivery_error_text(res),
1159                 show_alert=True,
1160             )
1161         return
1162
1163     # 4. Agar botning o'zida shaxsiy chatda bosilgan bo'lsa -> Shaxsiy chatga yetkazish
1164     await deliver_private_lesson(client, user_id=user_id, lesson_id=lesson_id, query_id=query_id)
1165
1166
1167
1168 async def _handle_admin_video_message(
1169     client: httpx.AsyncClient,
1170     message: Dict[str, Any],
1171     admin_id: int,
1172 ) -> bool:
1173     """Admin to'g'ridan-to'g'ri video yuborganida 1-bosishda dars yaratish oqimi."""
1174     video = message.get("video")
1175     if not video and message.get("document") and str((message.get("document") or {}).get("mime_type") or "").startswith("video/"):
1176         video = message.get("document")
1177
1178     if not video or not is_admin(admin_id):
1179         return False
1180
1181     store = get_store()
1182     file_id = video.get("file_id")
1183     file_unique_id = video.get("file_unique_id")
1184     duration = int(video.get("duration") or 0)
1185     width = video.get("width")
1186     height = video.get("height")
1187     file_size = int(video.get("file_size") or 0)
1188     caption = str(message.get("caption") or "").strip()
1189
1190     courses = await store.list_courses(published_only=True)
1191     active_course = courses[0] if courses else None
1192     course_id = active_course["id"] if active_course else None
1193
1194     existing_count = await store.count_lessons(course_id=course_id)
1195     if existing_count == 0 and active_course:
1196         from app.api.courses import resolve_course_modules
1197         existing_count = len([l for m in resolve_course_modules(active_course) for l in m.get("lessons", [])])
1198     next_num = existing_count + 1
1199
1200     if caption:
1201         if "dars" in caption.lower():
1202             suggested_title = caption
1203         else:
1204             suggested_title = f"{next_num}-dars. {caption}"
1205     else:
1206         suggested_title = f"{next_num}-dars"
1207
1208     session_id = secrets.token_hex(6)
1209     _pending_admin_videos[admin_id] = {
1210         "file_id": file_id,
1211         "video_file_id": file_id,
1212         "video_file_unique_id": file_unique_id,
1213         "duration": duration,
1214         "duration_seconds": duration,
1215         "width": width,
1216         "height": height,
1217         "file_size": file_size,
1218         "suggested_title": suggested_title,
1219         "course_id": course_id,
1220         "created_by": admin_id,
1221         "session_id": session_id,
1222     }
1223
1224     dur_str = f"{duration // 60:02d}:{duration % 60:02d}"
1225     size_mb = round(file_size / (1024 * 1024), 1)
1226

```

```

1227 text = (
1228     "📺 <b>Yangi video dars qabul qilindi!</b>\n\n"
1229     f"🕒 <b>Davomiyligi:</b> <code>{dur_str}</code> | 📄 <b>Hajmi:</b> <code>{size_mb} MB</code>\n\n"
1230     f"💡 <b>Taklif etilgan nom:</b>\n\n"
1231     f"👉 <b>{<_escape(suggested_title)></b>\n\n"
1232     "<i>Quyidagi tugmani 1 marta bosib darsni darhol saqlashingiz mumkin, "
1233     "yoki boshqa nom xohlasangiz, uni shunchaki yozib yuboring:</i>"
1234 )
1235
1236 keyboard = {
1237     "inline_keyboard": [
1238         [{"text": f"✅ «{suggested_title[:28]}» deb saqlash", "callback_data": f"admin:save_video:{session_id}"}],
1239         [{"text": f"❌ Bekor qilish", "callback_data": f"admin:cancel_video:{session_id}"}],
1240     ]
1241 }
1242
1243 await send_tg_message(client, admin_id, text, reply_markup=keyboard)
1244 return True
1245
1246
1247 async def _save_pending_admin_video(
1248     client: httpx.AsyncClient,
1249     admin_id: int,
1250     custom_title: Optional[str] = None,
1251     message_id_to_edit: Optional[int] = None,
1252     expected_session_id: Optional[str] = None,
1253 ) -> Optional[Dict[str, Any]]:
1254     """Kutilayotgan videoni bazaga va Mini App katalogiga sinxron saqlash."""
1255     video_data = _pending_admin_videos.get(admin_id)
1256     if not video_data:
1257         return None
1258     if expected_session_id and video_data.get("session_id") != expected_session_id:
1259         return None
1260     _pending_admin_videos.pop(admin_id, None)
1261
1262     store = get_store()
1263     title = (custom_title or video_data.get("suggested_title") or "Yangi dars").strip()
1264     course_id = video_data.get("course_id")
1265     if not course_id:
1266         courses = await store.list_courses(published_only=True)
1267         if courses:
1268             course_id = courses[0]["id"]
1269
1270     dur_sec = int(video_data.get("duration") or 0)
1271     dur_str = f"{dur_sec // 60:02d}:{dur_sec % 60:02d}"
1272     size_mb = round((video_data.get("file_size") or 0) / (1024 * 1024), 1)
1273
1274     lesson = await store.add_lesson({
1275         "title": title,
1276         "course_id": course_id,
1277         "video_file_id": video_data.get("video_file_id"),
1278         "video_file_unique_id": video_data.get("video_file_unique_id"),
1279         "duration": dur_str,
1280         "duration_seconds": dur_sec,
1281         "width": video_data.get("width"),
1282         "height": video_data.get("height"),
1283         "file_size": video_data.get("file_size"),
1284         "created_by": admin_id,
1285     })
1286
1287     lesson_id = lesson["id"]
1288
1289     # Mini App katalogiga va modullariga sinxronlash
1290     if course_id:
1291         course = await store.get_course(course_id)
1292         if course:
1293             modules = list(course.get("modules") or [])
1294             if not modules:
1295                 modules = [{"title": "01. Asosiy Video Darslar", "lessons": []}]
1296             modules[-1].setdefault("lessons", []).append({
1297                 "id": str(lesson_id),
1298                 "title": title,
1299                 "duration": dur_str,
1300                 "video_url": f"tg-file:{video_data.get('video_file_id')}",
1301                 "telegram_file_id": video_data.get("video_file_id"),
1302                 "file_id": video_data.get("video_file_id"),
1303                 "is_preview": False,
1304             })
1305             course["modules"] = modules
1306             course["lesson_count"] = sum(len(m.get("lessons", [])) for m in modules)
1307             await store.upsert_course(course)
1308
1309     target_id, target_title = await _get_effective_target_group(store, course_id)
1310     target_name = target_title if target_id else "Belgilanmagan (/set_group)"
1311
1312     confirmation_text = (
1313         f"📌 <b>{<_escape(title)></b> muvaffaqiyatli saqlandi!\n\n"
1314         f"🔖 <b>Dars ID:</b> <code>{lesson_id}</code>\n\n"

```

```

1315 f"🔔 <b>Davomiyiligi:</b> <code>{dur_str}</code> | 📄 <b>Hajmi:</b> <code>{size_mb}</code> MB</code>\n"
1316 f"🎯 <b>Target Guruh:</b> <b>{<_escape(target_name)></b>\n\n"
1317 "Endi darsni guruhga chiqarishingiz yoki yangi video yuklashingiz mumkin:"
1318 )
1319
1320 keyboard = {
1321     "inline_keyboard": [
1322         [{"text": "📌 Guruhga chiqarish", "callback_data": f"admin:publish:{lesson_id}"}],
1323         [
1324             {"text": "📺 Yana dars qo'shish", "callback_data": "admin:add_video"},
1325             {"text": "📖 Darslar ro'yxati", "callback_data": "admin:lessons"}
1326         ],
1327         [{"text": "⚙️ Admin Panel (Mini App)", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}],
1328     ]
1329 }
1330
1331 if message_id_to_edit:
1332     edit_res = await _telegram_call(
1333         client,
1334         "editMessageText",
1335         {"chat_id": admin_id, "message_id": message_id_to_edit, "text": confirmation_text, "parse_mode": "HTML", "reply_markup": keyboard}
1336     )
1337     if not edit_res.get("ok"):
1338         await send_tg_message(client, admin_id, confirmation_text, reply_markup=keyboard)
1339 else:
1340     await send_tg_message(client, admin_id, confirmation_text, reply_markup=keyboard)
1341
1342 return lesson
1343
1344
1345 async def _send_admin_lessons_list(client: httpx.AsyncClient, admin_id: int) -> None:
1346     """Admin uchun barcha darslar va ularni guruhga chiqarish tugmalari."""
1347     store = get_store()
1348     lessons = await store.list_lessons()
1349     if not lessons:
1350         await send_tg_message(
1351             client,
1352             admin_id,
1353             "📺 Hozircha saqlangan video darslar yo'q.\n\nVideo yuboring va 1-bosishda dars yarating!",
1354             {"inline_keyboard": [{"text": "📺 Video yuklash", "callback_data": "admin:add_video"}]}
1355         )
1356     return
1357
1358 text = f"📺 <b>Barcha Video Darslar ({len(lessons)} ta):</b>\n\n"
1359 rows = []
1360 for l in lessons[:15]:
1361     lid = l["id"]
1362     ltitle = l.get("title") or "Dars"
1363     dur = l.get("duration") or ""
1364     text += f"• <b>{<_escape(ltitle)></b> (<code>{dur}</code>) – ID: <code>{lid}</code>\n"
1365     rows.append([
1366         {"text": f"📌 #<code>{lid}</code> Guruhga chiqarish", "callback_data": f"admin:publish:{lid}"},
1367         {"text": f"📖 Sinab ko'rish", "callback_data": f"lesson:{lid}"},
1368     ])
1369
1370 rows.append([{"text": "📺 Yangi dars qo'shish", "callback_data": "admin:add_video"}])
1371 await send_tg_message(client, admin_id, text, reply_markup={"inline_keyboard": rows})
1372
1373
1374 async def _send_direct_lesson_video(
1375     client: httpx.AsyncClient,
1376     chat_id: int,
1377     user_id: int,
1378     lesson_id: str | int,
1379 ) -> None:
1380     """Talabaga dars videosini to'g'ridan-to'g'ri shaxsiy chatda himoyalangan holda yuborish (H1 xavfsizligi)."""
1381     await deliver_private_lesson(client, user_id=user_id, lesson_id=lesson_id, query_id=None)
1382
1383
1384 async def _handle_admin_callback(client: httpx.AsyncClient, callback_query: Dict[str, Any]) -> bool:
1385     data = str(callback_query.get("data") or "")
1386     query_id = callback_query.get("id", "")
1387     admin_tg_id = callback_query.get("from", {}).get("id")
1388
1389     # Admin video & darslar boshqaruvi
1390     if data.startswith("admin:"):
1391         if not is_admin(admin_tg_id):
1392             await _answer_callback(client, query_id, "Sizda admin huquqlari yo'q")
1393             return True
1394
1395         if data.startswith("admin:save_video"):
1396             parts = data.split(":")
1397             session_id = parts[2] if len(parts) > 2 else None
1398             pending = _pending_admin_videos.get(int(admin_tg_id))
1399             if not pending or not session_id or pending.get("session_id") != session_id:
1400                 await _answer_callback(client, query_id, "⚠️ Bu video so'rovi topilmadi yoki allaqachon eskirgan.", show_alert=True)
1401                 return True
1402             msg = callback_query.get("message") or {}

```

```

1403     msg_id = msg.get("message_id")
1404     await _answer_callback(client, query_id, "✅ Dars saqlanmoqda...")
1405     await _save_pending_admin_video(client, int(admin_tg_id), message_id_to_edit=msg_id, expected_session_id=session_id)
1406     return True
1407
1408     if data.startswith("admin:cancel_video"):
1409         parts = data.split(":")
1410         session_id = parts[2] if len(parts) > 2 else None
1411         pending = _pending_admin_videos.get(int(admin_tg_id))
1412         if not pending or not session_id or pending.get("session_id") != session_id:
1413             await _answer_callback(client, query_id, "⚠️ Bu video so'rovi topilmadi yoki allaqachon eskirgan.", show_alert=True)
1414             return True
1415         _pending_admin_videos.pop(int(admin_tg_id), None)
1416         await _answer_callback(client, query_id, "Video bekor qilindi.")
1417         msg = callback_query.get("message") or {}
1418         msg_id = msg.get("message_id")
1419         if msg_id:
1420             await _telegram_call(
1421                 client,
1422                 "editMessageText",
1423                 {"chat_id": admin_tg_id, "message_id": msg_id, "text": "❌ Video yuklash bekor qilindi.", "parse_mode": "HTML"}
1424             )
1425             return True
1426
1427     if data.startswith("admin:publish:"):
1428         lid = data.split(":", 2)[2]
1429         await _answer_callback(client, query_id, "Guruhga chiqarilmoqda...")
1430         await publish_lesson_to_group(client, lid, int(admin_tg_id))
1431         return True
1432
1433     if data == "admin:lessons":
1434         await _answer_callback(client, query_id)
1435         await _send_admin_lessons_list(client, int(admin_tg_id))
1436         return True
1437
1438     if data == "admin:add_video":
1439         await _answer_callback(client, query_id)
1440         await send_tg_message(
1441             client,
1442             int(admin_tg_id),
1443             "📺 <b>Yangi dars videosini yuboring.</b>\n\n"
1444             "Video yuborganingizda tizim uning hajmi va davomiyligini tekshirib, 1-bosishda dars yaratish imkonini beradi."
1445         )
1446         return True
1447
1448     return False
1449
1450     if not (data.startswith("approve_") or data.startswith("reject_")):
1451         return False
1452
1453     if admin_tg_id not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1454         await _answer_callback(client, query_id, "Sizda admin huquqlari yo'q")
1455         return True
1456
1457     order_id = data.split("_", 1)[1]
1458     profile = settings.ADMIN_PROFILES.get(admin_tg_id, {})
1459     admin_name = profile.get("name") or callback_query.get("from", {}).get("first_name") or "Superadmin"
1460     approved = data.startswith("approve_")
1461     ok, result = (
1462         await approve_purchase(order_id, admin_name)
1463         if approved
1464         else await reject_purchase(order_id, admin_name)
1465     )
1466     await _answer_callback(client, query_id, ("✅ " if ok else "⚠️ ") + result)
1467     if not ok:
1468         return True
1469
1470     message = callback_query.get("message") or {}
1471     chat = message.get("chat") or {}
1472     current_text = str(message.get("caption") or message.get("text") or "")
1473     status_line = "✅ <b>TO'LOV TASDIQLANDI</b>" if approved else "❌ <b>TO'LOV RAD ETILDI</b>"
1474     # Telegram eski captionni tekis matn (entities qo'llanmagan holda) qaytaradi.
1475     # Uni parse_mode=HTML bilan qayta yuborishdan oldin escape qilish shart –
1476     # talaba ismi/izohidagi & yoki < "can't parse entities" xatosini keltirib,
1477     # chek kartasiga status qatori umuman qo'shilmay qolardi.
1478     updated = (
1479         f"_{escape(current_text)}\n\n—————\n"
1480         f"_{status_line}\n"
1481         f"👤 <b>Admin:</b> { _escape(admin_name)}"
1482     )[:1024]
1483     method = "editMessageCaption" if message.get("caption") is not None else "editMessageText"
1484     field = "caption" if method == "editMessageCaption" else "text"
1485     edit_result = await _telegram_call(
1486         client,
1487         method,
1488         {"chat_id": chat.get("id"), "message_id": message.get("message_id"), field: updated, "parse_mode": "HTML"},
1489     )
1490     if not edit_result.get("ok"):

```

```

1491 # Tahrir o'tmasa (masalan, caption 1024 belgi chekkasida kesilganda ham
1492 # xato qaytishi mumkin) – qarorni alohida xabar sifatida yozamiz.
1493 await send_tg_message(
1494     client,
1495     int(chat.get("id") or 0) or int(admin_tg_id or 0),
1496     f"{status_line}\n"
1497     f"👤 <b>Admin:</b> { _escape(admin_name)}\n"
1498     f"🛒 <b>Buyurtma:</b> <code>{_escape(order_id)}</code>",
1499 )
1500 return True
1501
1502
1503 async def handle_callback_query(client: httpx.AsyncClient, callback_query: Dict[str, Any]) -> None:
1504     if await _handle_admin_callback(client, callback_query):
1505         return
1506
1507     query_id = callback_query.get("id", "")
1508     data = str(callback_query.get("data") or "")
1509     tg_user = callback_query.get("from") or {}
1510     chat_id = (callback_query.get("message") or {}).get("chat", {}).get("id") or tg_user.get("id")
1511     user = await _ensure_user(tg_user)
1512     if not user or not chat_id:
1513         await _answer_callback(client, query_id, "Foydalanuvchi aniqlanmadi")
1514         return
1515
1516     if data.startswith("lesson:"):
1517         lesson_id = data.split(":", 1)[1]
1518         await handle_student_ephemeral_lesson(client, callback_query, lesson_id)
1519         return
1520
1521     if data == "noop":
1522         await _answer_callback(client, query_id)
1523         return
1524     if data.startswith("course:"):
1525         try:
1526             index = int(data.split(":", 1)[1])
1527         except ValueError:
1528             await _answer_callback(client, query_id, "Kurs topilmadi")
1529             return
1530         await _answer_callback(client, query_id)
1531         await show_course_card(client, int(chat_id), index)
1532         return
1533     if data.startswith("pay:"):
1534         await _answer_callback(client, query_id)
1535         await _start_checkout(client, int(chat_id), user, data.split(":", 1)[1])
1536         return
1537     if data == "checkout:cancel":
1538         _checkout_sessions.pop(int(user["telegram_id"]), None)
1539         await _answer_callback(client, query_id, "To'lov bekor qilindi")
1540         await send_tg_message(client, int(chat_id), "To'lov jarayoni bekor qilindi. Istalgan payt kursni qayta tanlashingiz mumkin.")
1541         return
1542     if data == "payments":
1543         await _answer_callback(client, query_id)
1544         await _send_payment_info(client, int(chat_id))
1545         return
1546     if data == "help":
1547         await _answer_callback(client, query_id)
1548         await _send_help(client, int(chat_id))
1549         return
1550     await _answer_callback(client, query_id, "Bu tugma endi faol emas")
1551
1552
1553 async def handle_chat_join_request(client: httpx.AsyncClient, join_request: Dict[str, Any]) -> None:
1554     """Link va xaridor akkaunti mos kelgandagina join requestni tasdiqlaydi.
1555
1556     Tasdiqlangach havola darhol bekor qilinadi (bir martalik link): uni boshqa
1557     odamga qayta yuborib yoki eski zayavkani qayta bosib kirib bo'lmaydi.
1558     """
1559     chat = join_request.get("chat") or {}
1560     requester = join_request.get("from") or {}
1561     chat_id = chat.get("id")
1562     telegram_id = requester.get("id")
1563     invite = join_request.get("invite_link") or {}
1564     invite_link = invite.get("invite_link") if isinstance(invite, dict) else None
1565     if not chat_id or not telegram_id:
1566         return
1567
1568     authorized, purchase = await is_join_request_authorized(int(chat_id), int(telegram_id), invite_link)
1569     if not authorized:
1570         decline_res = await _telegram_call(client, "declineChatJoinRequest", {"chat_id": chat_id, "user_id": telegram_id})
1571         if not decline_res.get("ok"):
1572             logger.warning("declineChatJoinRequest muvaffaqiyatsiz bo'ldi: user_id=%s, res=%s", telegram_id, decline_res)
1573         await send_tg_message(
1574             client,
1575             int(telegram_id),
1576             "🚫 <b>Kanalga kirish rad etildi.</b> Bu havola boshqa akkautga berilgan, allaqachon ishlatilgan yoki amal qilish muddati tugagan. "
1577             "Kursni o'z akkauntingizdan sotib oling.",
1578             {"inline_keyboard": [{"text": "🛒 Mini Appni ochish", "web_app": {"url": settings.WEBAPP_URL}}]}),

```

```

1579         )
1580         return
1581
1582     # Authorized: zayavkani tasdiqlash
1583     result = await _telegram_call(
1584         client, "approveChatJoinRequest", {"chat_id": chat_id, "user_id": telegram_id}
1585     )
1586     if not result.get("ok"):
1587         err_code = result.get("error_code")
1588         desc = result.get("description", "")
1589         logger.error("approveChatJoinRequest failed for user %s: code=%s desc=%s", telegram_id, err_code, desc)
1590         # Failure recovery: link bekor qilinmaydi, talaba qayta urinishi mumkin
1591         await send_tg_message(
1592             client,
1593             int(telegram_id),
1594             "🚨 <b>Guruhga qo'shishda Telegram tomonidan vaqtinchalik xatolik yuz berdi.</b>\n"
1595             "Iltimos, havolani qayta bosib yoki birozdan so'ng urinib ko'ring.",
1596         )
1597         return
1598
1599     # Muvaqqafiyatli qabul qilinganidan keyingina havola bekor qilinadi
1600     if invite_link:
1601         await revoke_join_request_link(client, int(chat_id), invite_link)
1602
1603     title = _escape((purchase or {}).get("course_title") or chat.get("title") or "kurs")
1604     keyboard_rows = [
1605         [{"text": "📌 Kursni Mini Appda ochish", "web_app": {"url": settings.WEBAPP_URL}}],
1606     ]
1607
1608     await send_tg_message(
1609         client,
1610         int(telegram_id),
1611         f"👋 <b>Xush kelibsiz!</b> Sizning <b>{title}</b> guruhiga a'zoligingiz tasdiqlandi.\n\n"
1612         "✅ <b>Darslarni tomosha qilish:</b>\n"
1613         "1. Guruhdagi istalgan dars e'lonidagi <b>[📺 Darsni ko'rish]</b> tugmasini bosib – video faqat sizning ekraningizda ochiladi!\n"
1614         "2. Yoki to'g'ridan-to'g'ri Mini App orqali barcha darslarni tartib bilan o'rganishingiz mumkin.",
1615         {"inline_keyboard": keyboard_rows}
1616     )
1617
1618
1619 async def handle_my_chat_member(client: httpx.AsyncClient, update: Dict[str, Any]) -> None:
1620     """Bot kanalga admin bo'lganda superadminlarga kanal ID sini yuboradi.
1621     Guruhga qo'shilganda esa qisqa tanishtiruv yuboradi."""
1622     chat = update.get("chat") or {}
1623     old_status = (update.get("old_chat_member") or {}).get("status")
1624     new_status = (update.get("new_chat_member") or {}).get("status")
1625
1626     if chat.get("type") in {"group", "supergroup"}:
1627         if new_status in {"member", "administrator"} and old_status in {"left", "kicked"}:
1628             await _send_group_intro(client, int(chat.get("id") or 0))
1629             return
1630
1631     if new_status not in {"administrator", "creator"} or old_status == new_status:
1632         return
1633     if chat.get("type") not in {"channel", "supergroup"}:
1634         return
1635
1636     text = (
1637         "📌 <b>Yangi dars kanali ulandi</b>\n\n"
1638         f"🌟 Nomi: <b>{_escape(chat.get('title') or 'Kanal')}</b>\n\n"
1639         f"🆔 Kanal ID: <code>{chat.get('id')}</code>\n\n"
1640         "Admin paneldagi kurs tahrirlash oynasiga shu ID ni <code>telegram_channel_id</code> sifatida kiriting."
1641     )
1642     for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1643         await send_tg_message(client, admin_id, text)
1644
1645
1646 async def _send_help(client: httpx.AsyncClient, chat_id: int) -> None:
1647     await send_tg_message(
1648         client,
1649         chat_id,
1650         "🧠 <b>Kreativ AI – Yordam markazi</b>\n\n"
1651         "To'lov, kurslar, yopiq kanallar yoki takliflar bo'yicha superadminlarga yozing:\n\n"
1652         "👤 <b>Yaxshi Bola</b> – @yomonboIa\n"
1653         "👤 <b>Zuhra Olimova</b> – @sokin_notalar",
1654         {"inline_keyboard": [
1655             [{"text": "📌 Kreativ AI Mini App", "web_app": {"url": settings.WEBAPP_URL}},
1656             [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}]
1657         ]},
1658     )
1659
1660
1661 async def _send_welcome(client: httpx.AsyncClient, chat_id: int, tg_user: Dict[str, Any]) -> None:
1662     first_name = _escape(tg_user.get("first_name") or "Do'stim")
1663     is_admin = tg_user.get("id") in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]
1664     text = (
1665         f"Assalomu alaykum, <b>{first_name}</b>! 🌟\n\n"
1666         "<b>Kreativ AI</b> – Sun'iy intellekt, dizayn va zamonaviy kasblar platformasiga xush kelibsiz!\n\n"

```

```

1667         "🔥 <b>Imkoniyatlar:</b>\n"
1668         "• 📺 Yuqori sifatli amaliy kurslar\n"
1669         "• 📄 Tekshiriluvchi QR-kodli sertifikatlar\n"
1670         "• 🏔 Himoyalangan yopiq dars kanallari\n"
1671         "Darslarni boshlash uchun quyidagi tugmani bosing 📌"
1672     )
1673     keyboard_rows = [
1674         [{"text": "📌 Platformani Ochish (Mini App)", "web_app": {"url": settings.WEBAPP_URL}}],
1675         [{"text": "📺 Kurslar Katalogi", "callback_data": "course:0"}, {"text": "💰 To'lov Rekvizitlari", "callback_data": "payments"}],
1676         [{"text": "👤 Admin Yordami", "callback_data": "help"}],
1677     ]
1678     if is_admin:
1679         keyboard_rows.append([{"text": "🔑 Superadmin Boshqaruv Paneli", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}])
1680     keyboard = {"inline_keyboard": keyboard_rows}
1681
1682     banner_url = getattr(settings, "WELCOME_BANNER_URL", "")
1683     if not banner_url:
1684         try:
1685             courses = await get_store().list_courses(published_only=True)
1686             banner_url = str(courses[0].get("cover_url") or "") if courses else ""
1687         except Exception:
1688             banner_url = ""
1689     banner_url = _resolve_media_url(banner_url)
1690     if banner_url.startswith(("https://", "http://")):
1691         result = await send_tg_photo(client, chat_id, banner_url, text, keyboard)
1692         if result.get("ok"):
1693             return
1694     await send_tg_message(client, chat_id, text, keyboard)
1695
1696
1697 async def _send_stats(client: httpx.AsyncClient, chat_id: int, user_id: int) -> None:
1698     if user_id not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1699         await send_tg_message(client, chat_id, "❌ Bu buyruq faqat superadminlar uchun.")
1700         return
1701     store = get_store()
1702     stats = await store.revenue_stats()
1703     pending = await store.list_purchases(status="pending_approval", limit=1000)
1704     active_courses = await store.list_courses(published_only=True)
1705     # Ayrim adminlarga (masalan Zuhra Olimova) talabalar soni ko'rsatilmaydi
1706     students_line = ""
1707     if user_id not in settings.STATS_STUDENTS_HIDDEN_IDS:
1708         students_line = f"👥 Talabalar: <b>{await store.count_users()} ta</b>\n"
1709     text = (
1710         "📊 <b>Kreativ AI – Platforma Statistikasi</b>\n\n"
1711         f"🔥 Jami tushum: <b>{_uzs(stats.get('total_revenue'))}</b>\n"
1712         f"💰 Oylik tushum: <b>{_uzs(stats.get('monthly_revenue'))}</b>\n"
1713         f"{students_line}"
1714         f"📺 Faol kurslar: <b>{len(active_courses)} ta</b>\n"
1715         f"📄 Kutilayotgan cheklar: <b>{len(pending)} ta</b>"
1716     )
1717     await send_tg_message(
1718         client, chat_id, text,
1719         {"inline_keyboard": [{"text": "🔑 Admin Panelni Ochish", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}]}],
1720     )
1721
1722
1723
1724 async def _handle_unknown_text(client: httpx.AsyncClient, chat_id: int) -> None:
1725     """Matn ko'rinishidagi barcha xabarlariga yo'naltiruvchi javob (AI chat olib tashlangan)."""
1726     await send_tg_message(
1727         client,
1728         chat_id,
1729         "📺 <b>Kurslar platformasi</b>\n\n"
1730         "Kurslarni ko'rish uchun katalogni oching yoki to'lov uchun rekvizitlarni so'rang. "
1731         "Savollaringiz bo'lsa, adminlar /help bo'limida.",
1732         {"inline_keyboard": [
1733             [{"text": "📺 Kurslar Katalogi", "callback_data": "course:0"}, {"text": "💰 To'lov Rekvizitlari", "callback_data": "payments"}],
1734             [{"text": "📌 Mini App", "web_app": {"url": settings.WEBAPP_URL}}],
1735         ]},
1736     )
1737
1738
1739 async def _get_bot_id(client: httpx.AsyncClient) -> Optional[int]:
1740     """Botning o'z ID sini bir marta olib keshlaydi (guruhda reply-to-bot aniqlash uchun)."""
1741     global BOT_ID
1742     if BOT_ID is None:
1743         data = await _telegram_call(client, "getMe", {})
1744         BOT_ID = (data.get("result") or {}).get("id")
1745     return BOT_ID
1746
1747
1748 def _group_ai_allowed(chat_id: int) -> bool:
1749     """Har bir guruhda AI javoblari: kamida 15 soniya oraliq va soatiga 8 tadan."""
1750     now = time.time()
1751     st = _GROUP_AI_STATE.setdefault(chat_id, {"times": [], "last": 0.0})
1752     st["times"] = [t for t in st["times"] if now - t < 3600]
1753     if now - st["last"] < 15:
1754         return False

```

```

1755     if len(st["times"]) >= 8:
1756         return False
1757     return True
1758
1759
1760 def _group_ai_mark(chat_id: int) -> None:
1761     now = time.time()
1762     st = _GROUP_AI_STATE.setdefault(chat_id, {"times": [], "last": 0.0})
1763     st["last"] = now
1764     st["times"].append(now)
1765
1766
1767 async def _send_group_intro(client: httpx.AsyncClient, chat_id: int) -> None:
1768     """Bot guruhga qo'shilganda qisqa tanishtiruv."""
1769     if not chat_id:
1770         return
1771     bot_user = settings.BOT_USERNAME.lstrip("@")
1772     text = (
1773         "👋 <b>Assalomu alaykum!</b> Men – <b>Kreativ AI</b> o'quv platformasining yordamchisan.\n\n"
1774         "👨‍💻 Guruhda savolingizni <b>@kreativaibot</b> deb yozib bering – AI yordamchim javob beradi "
1775         "va kasb o'rganish bo'yicha yo'l ko'rsatadi.\n\n"
1776         "📚 Kurslar, narxlar va chegirmalar haqida bimalol so'rang!"
1777     )
1778     keyboard = {
1779         "inline_keyboard": [
1780             [{"text": "👉 Kurslarni ko'rish", "url": f"https://t.me/{bot_user}?start=catalog"}],
1781             [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}],
1782         ]
1783     }
1784     await send_tg_message(client, chat_id, text, keyboard, protect_content=False)
1785
1786
1787 async def _build_catalog_context(store) -> str:
1788     """AI ga beriladigan katalog konteksti – faqat real ma'lumotlar (raqam o'ylab topish taqiqlanadi)."""
1789     try:
1790         courses = await store.list_courses(published_only=True)
1791     except Exception:
1792         courses = []
1793     lines: list = []
1794     for c in courses[:8]:
1795         try:
1796             pricing = await course_pricing(store, c)
1797         except Exception:
1798             pricing = {'discount_active': False, "discount_spots_left": None, "final_price": c.get("price")}
1799         line = (
1800             f"• «{c.get('title')}» – kategoriya: {c.get('category')}, "
1801             f"{int(c.get('lesson_count') or 0)} ta dars, narxi {_uzs(pricing['final_price'])}, "
1802             f"ustoz: {c.get('instructor_name')}"
1803         )
1804         if pricing["discount_active"] and c.get("discount_percent"):
1805             line += (
1806                 f", 🟡 -{int(c['discount_percent'])}% chegirma birinchi {int(c.get('discount_limit') or 0)} kishi uchun "
1807                 + (f"({int(pricing['discount_spots_left'])} ta joy qoldi)" if pricing.get("discount_spots_left") is not None else "")
1808             )
1809         lines.append(line)
1810     return "\n".join(lines) or "(katalog hozircha bo'sh)"
1811
1812
1813 _GROUP_SYSTEM_PROMPT = (
1814     "Siz – «Kreativ AI» onlayn kurslar platformasining Telegram guruh yordamchisiz. "
1815     "Vazifangiz: guruhdagi savollarga qisqa, do'stona va foydali javob berish hamda tabiiy tarzda "
1816     "platformadagi amaliy kurslarga qiziqish uyg'otish.\n\n"
1817     "Qoidalar:\n"
1818     "1. O'zbek tilida, sodda va samimiy ohangda yozing. Javob 1-4 gap + zarur bo'lsa 2-3 ta banddan oshmasin.\n"
1819     "2. Narx, chegirma, darslar soni – FAQAT katalogdan oling. Hech qachon raqam, aksiya yoki kurs "
1820     "nomini o'ylab chiqmang. Katalogda yo'q kursni tavsiya qilmang.\n"
1821     "3. Savol o'qish/kasb/ko'nikmaga aloqador bo'lsa – eng mos 1 ta kursni nomi va narxi bilan tavsiya qiling.\n"
1822     "4. Savol butunlay boshqa mavzuda bo'lsa – qisqa foydali javob bering va suhbatni kasb o'rganish "
1823     "mavzusiga tabiiy bog'lang.\n"
1824     "5. Spam, qattiq reklama ohangi bo'lmasin. Siyosiy, diniy va ta'qibli mavzularda betaraf qisqa javob bering.\n"
1825     "6. Javob oxirida 1 marta yumshoq CTA bo'lsin (masalan: 'Batafsil – Mini Appda 📲').\n"
1826     "7. Maksimal ~500 belgi. Emoji dan o'rnida foydalaning (1-3 dona)."
```

```

1827 )
1828
1829
1830 async def _group_ai_answer(
1831     client: httpx.AsyncClient,
1832     chat_id: int,
1833     reply_to_message_id: Optional[int],
1834     question: str,
1835     sender_name: str,
1836 ) -> None:
1837     """Guruhdagi savolga AI javob beradi: katalog konteksti + sotuvga yo'naltirilgan persona."""
1838     store = get_store()
1839     catalog = await _build_catalog_context(store)
1840     messages = [
1841         {"role": "system", "content": _GROUP_SYSTEM_PROMPT},
1842         {

```

```

1843         "role": "user",
1844         "content": (
1845             f"Guruh a'zosi {sender_name} quyidagi savolni yozdi:\n"
1846             f"\n{question[:600]}\n\n"
1847             f"Platformaning joriy katalogi:\n{catalog}"
1848         ),
1849     },
1850 ]
1851 await send_tg_chat_action(client, chat_id, "typing")
1852 answer = await asyncio.to_thread(call_openrouter_api, messages)
1853 if not answer:
1854     top = "\n".join(catalog.splitlines()[:3]) if catalog and not catalog.startswith("(") else ""
1855     answer = (
1856         "👋 Men hozircha savolga to'liq javob bera olmayaman, lekin Kreativ AI platformasidagi "
1857         "amaliy kurslar bilan tanishtiraman:\n"
1858         f"{top}\n\n"
1859         "Batafsil narx va chegirmalar Mini Appda 📲"
1860     )
1861 answer = answer.strip()
1862 if len(answer) > 1200:
1863     answer = answer[:1190].rstrip() + "..."
1864
1865 bot_user = settings.BOT_USERNAME.lstrip("@")
1866 payload: Dict[str, Any] = {
1867     "chat_id": chat_id,
1868     "text": _escape(answer),
1869     "disable_web_page_preview": True,
1870     "reply_markup": {"inline_keyboard": [
1871         [{"text": "📖 Kurslarni ko'rish", "url": f"https://t.me/{bot_user}?start=catalog"}],
1872         [{"text": "📁 Kurslar Katalogi", "callback_data": "course:0"}],
1873     ]},
1874 }
1875 if reply_to_message_id:
1876     payload["reply_to_message_id"] = reply_to_message_id
1877     payload["allow_sending_without_reply"] = True
1878 await _telegram_call(client, "sendMessage", payload)
1879
1880
1881 def _group_should_answer(message: Dict[str, Any], text: str) -> bool:
1882     """Guruhda bot faqat haqiqiy murojaatlarga javob beradi (spam qilmaydi):
1883     @mention, bot xabariga javob, yoki sotuvga aloqador so'zli savol."""
1884     lowered = text.lower()
1885     bot_username = settings.BOT_USERNAME.strip().lower()
1886     # BOT_USERNAME sozlanmagan (bo'sh) bo'lsa "@" tekshiruvi o'chiriladi -
1887     # aks holda f"@{' }" -> "@" har qanday @li xabarga match qilib, bot spam qilardi.
1888     if bot_username and f"@{bot_username}" in lowered:
1889         return True
1890     reply_from = (message.get("reply_to_message") or {}).get("from") or {}
1891     if BOT_ID and reply_from.get("id") == BOT_ID:
1892         return True
1893     if bot_username and reply_from.get("username") and reply_from["username"].strip().lower() == bot_username:
1894         return True
1895     if "?" in text and any(k in lowered for k in _GROUP_SALES_KEYWORDS):
1896         return True
1897     return False
1898
1899
1900 async def _handle_group_message(client: httpx.AsyncClient, message: Dict[str, Any]) -> None:
1901     """Guruh/supergrup xabarлари: komandalar + AI savol-javob. DB ga user yozilmaydi."""
1902     chat = message.get("chat") or {}
1903     chat_id = int(chat.get("id") or 0)
1904     tg_user = message.get("from") or {}
1905     if not chat_id or tg_user.get("is_bot"):
1906         return
1907
1908     text = str(message.get("text") or "").strip()
1909     if text.startswith("/"):
1910         command = text.split(maxsplit=1)[0].split("@", 1)[0].lower()
1911         if command == "/start":
1912             await _send_group_intro(client, chat_id)
1913         elif command in {"/kurslar", "/courses"}:
1914             await show_course_card(client, chat_id, 0)
1915         elif command in {"/tolov", "/payment", "/payments"}:
1916             await _send_payment_info(client, chat_id)
1917         elif command in {"/help", "/contact"}:
1918             await _send_help(client, chat_id)
1919         elif command in {"/stats", "/admin"}:
1920             tg_id = int(tg_user.get("id") or 0)
1921             if is_admin(tg_id):
1922                 # Statistika (tushum, talabalar soni) maxfiy – guruhga yozilmaydi,
1923                 # faqat adminning o'ziga shaxsiy chatda yuboriladi.
1924                 await _send_stats(client, tg_id, tg_id)
1925             elif command in {"/set_group", "/setgroup"}:
1926                 tg_id = int(tg_user.get("id") or 0)
1927                 if is_admin(tg_id):
1928                     store = get_store()
1929                     chat_title = chat.get("title") or f"Guruh {chat_id}"
1930                     await store.set_setting("target_group_id", str(chat_id))

```

```

1931         await store.set_setting("target_group_title", chat_title)
1932         settings.TARGET_GROUP_ID = str(chat_id)
1933         await send_tg_message(
1934             client,
1935             chat_id,
1936             f"✅ <b>Ushbu guruh asosiy darslar guruhi sifatida biriktirildi!</b>\n\n"
1937             f"📄 Guruh ID: <code>{chat_id}</code>\n"
1938             f"🌸 Nomi: <b>{_escape(chat_title)}</b>\n\n"
1939             "Endi yangi darslar va e'lonlar ushbu guruhga chiqariladi.",
1940         )
1941     return
1942
1943 if not text or not _group_should_answer(message, text):
1944     return
1945 if not _group_ai_allowed(chat_id):
1946     return
1947 _group_ai_mark(chat_id)
1948 sender_name = _escape(tg_user.get("first_name") or "Do'stim")
1949 try:
1950     await _group_ai_answer(client, chat_id, message.get("message_id"), text, sender_name)
1951 except Exception:
1952     logger.exception("Guruh AI javobida xato (chat %s)", chat_id)
1953
1954
1955 async def handle_tg_update(client: httpx.AsyncClient, update: Dict[str, Any]) -> None:
1956     if "chat_join_request" in update:
1957         await handle_chat_join_request(client, update["chat_join_request"])
1958         return
1959     if "my_chat_member" in update:
1960         await handle_my_chat_member(client, update["my_chat_member"])
1961         return
1962
1963     # Bloklangan foydalanuvchilar bot bilan muloqot qila olmaydi (adminlar bundan mustasno)
1964     raw_tg_id = None
1965     if "callback_query" in update:
1966         raw_tg_id = ((update["callback_query"].get("from")) or {}).get("id")
1967     elif update.get("message"):
1968         raw_tg_id = ((update["message"].get("from")) or {}).get("id")
1969     if raw_tg_id:
1970         try:
1971             tg_id_int = int(raw_tg_id)
1972         except (TypeError, ValueError):
1973             tg_id_int = None
1974         if tg_id_int is not None and tg_id_int not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1975             try:
1976                 blocked = await get_store().is_user_blocked(tg_id_int)
1977             except Exception as exc:
1978                 logger.error("Foydalanuvchi bloklanganligini tekshirishda DB xatosi (%s): %s", tg_id_int, exc)
1979                 # Xavfsizlik: DB xatosi yuz bersa nozik amallar (callback, to'lov/chek, start buyruqlari) to'xtatiladi (fail-closed)
1980                 if "callback_query" in update:
1981                     await _answer_callback(client, update["callback_query"].get("id"), "Tizimda vaqtinchalik xatolik. Qaytadan urinib ko'ring.", show_alert=True)
1982                     return
1983                 msg = update.get("message") or {}
1984                 if msg.get("photo") or str(msg.get("text") or "").startswith("/"):
1985                     await send_tg_message(client, tg_id_int, "🚫 Tizimda vaqtinchalik xatolik. Iltimos, birozdan so'ng qayta urinib ko'ring.")
1986                     return
1987                 blocked = False
1988
1989             if blocked:
1990                 if "callback_query" in update:
1991                     await _answer_callback(client, update["callback_query"].get("id"), "Hisobingiz bloklangan.", show_alert=True)
1992                     return
1993
1994     if "callback_query" in update:
1995         await handle_callback_query(client, update["callback_query"])
1996         return
1997
1998     message = update.get("message")
1999     if not message:
2000         return
2001     tg_user = message.get("from") or {}
2002     chat = message.get("chat") or {}
2003     chat_id = (chat.get("id"))
2004     if not tg_user or not chat_id:
2005         return
2006
2007     # Guruh/supergrup xabarlar: bot faqat murojaatlarga javob beradi, DB ga
2008     # har bir guruh a'zosi yozilmaydi va chek/FSM oqimi faqat privat chatga xos.
2009     if chat.get("type") in {"group", "supergroup"}:
2010         await _handle_group_message(client, message)
2011         return
2012
2013     user = await _ensure_user(tg_user)
2014     if not user:
2015         return
2016
2017     admin_tg_id = int(user.get("telegram_id") or 0)
2018     has_video = bool(

```

```

2019     message.get("video")
2020     or (message.get("document") and str((message.get("document") or {}).get("mime_type") or "").startswith("video/"))
2021 )
2022 if has_video and is_admin(admin_tg_id):
2023     await _handle_admin_video_message(client, message, admin_tg_id)
2024     return
2025
2026 if await _handle_receipt_photo(client, message, user):
2027     return
2028
2029 text = str(message.get("text") or "").strip()
2030 if not text:
2031     if _checkout_sessions.get(int(user["telegram_id"])):
2032         await send_tg_message(client, int(chat_id), "❌ Chekni <b>rasm</b> sifatida yuboring. Hujjat yoki oddiy xabar qabul qilinmaydi.")
2033     return
2034
2035 # Kutilayotgan video bo'lsa va admin maxsus nom yozsa
2036 if admin_tg_id in _pending_admin_videos and not text.startswith("/"):
2037     await _save_pending_admin_video(client, admin_tg_id, custom_title=text)
2038     return
2039
2040 command = text.split(maxsplit=1)[0].split("@", 1)[0].lower()
2041 if command == "/start":
2042     payload = text.split(maxsplit=1)[1].strip() if len(text.split(maxsplit=1)) > 1 else ""
2043     if payload.lower().startswith("lesson_"):
2044         lid = payload.split("_", 1)[1].strip()
2045         await _send_direct_lesson_video(client, int(chat_id), int(user["telegram_id"]), lid)
2046         return
2047     if payload.lower().startswith("pay_"):
2048         cid = payload.split("_", 1)[1].strip()
2049         await _start_checkout(client, int(chat_id), user, cid)
2050         return
2051     if payload.lower() in {"catalog", "courses", "kurslar"}:
2052         await show_course_card(client, int(chat_id), 0)
2053         return
2054     if payload.lower().startswith("ref_"):
2055         from app.services.promos import link_referral
2056         ok, message, bonus = await link_referral(get_store(), user["id"], payload)
2057         if ok:
2058             await _send_welcome(client, int(chat_id), tg_user)
2059             await send_tg_message(
2060                 client,
2061                 int(chat_id),
2062                 "📌 <b>Do'stingiz taklifi qabul qilindi!</b>\n\n"
2063                 f"Sizga bir martalik <b>{int(bonus['percent'])}%</b> promokod berildi:\n"
2064                 f"<code>{bonus['code']}</code>\n\n"
2065                 "Kurs sotib olishda «Promokod» maydoniga shu kodni yozing.",
2066                 {
2067                     "inline_keyboard": [
2068                         [{"text": "👉 Kurslarni ko'rish", "web_app": {"url": settings.WEBAPP_URL}}]
2069                     ],
2070                 },
2071             )
2072             return
2073         await _send_welcome(client, int(chat_id), tg_user)
2074 elif command in {"/kurslar", "/courses"}:
2075     await show_course_card(client, int(chat_id), 0)
2076 elif command in {"/tolov", "/payment", "/payments"}:
2077     await _send_payment_info(client, int(chat_id))
2078 elif command in {"/stats", "/admin"}:
2079     await _send_stats(client, int(chat_id), int(user["telegram_id"]))
2080 elif command in {"/darslar", "/lessons"}:
2081     if is_admin(user["telegram_id"]):
2082         await _send_admin_lessons_list(client, int(chat_id))
2083     else:
2084         await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat adminlar uchun.")
2085 elif command in {"/set_group", "/setgroup"}:
2086     if is_admin(user["telegram_id"]):
2087         parts = text.split(maxsplit=2)
2088         if len(parts) > 1:
2089             target_id = parts[1].strip()
2090             target_title = parts[2].strip() if len(parts) > 2 else f"Guruh {target_id}"
2091             store = get_store()
2092             await store.set_setting("target_group_id", target_id)
2093             await store.set_setting("target_group_title", target_title)
2094             settings.TARGET_GROUP_ID = target_id
2095             await send_tg_message(
2096                 client,
2097                 int(chat_id),
2098                 f"✅ <b>Target Guruh muvaffaqiyatli saqlandi!</b>\n\n"
2099                 f"📌 ID: <code>{target_id}</code>\n\n"
2100                 f"👉 Nomi: <b>{_escape(target_title)}</b>"
2101             )
2102         else:
2103             await send_tg_message(
2104                 client,
2105                 int(chat_id),
2106                 "❌ <b>Guruhni birlashtirish uchun:</b>\n\n"
2107                 "1. Guruh ichida <code>/set_group</code> yozing, yoki\n"
2108                 "2. Shaxsiy chatda: <code>/set_group &lt;group_id&gt; [guruh nomi]</code> ko'rinishida yuboring."
2109             )
2110     else:

```

```

2107         await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat adminlar uchun.")
2108     elif command in {"/hisobot", "/report"}:
2109         if is_admin(user["telegram_id"]):
2110             await _send_daily_report(client, int(chat_id))
2111         else:
2112             await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat superadminlar uchun.")
2113     elif command == "/backup":
2114         if is_admin(user["telegram_id"]):
2115             await send_tg_message(client, int(chat_id), "📦 Zaxira tayyorlanmoqda...")
2116             await _send_backup(client, int(chat_id))
2117         else:
2118             await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat superadminlar uchun.")
2119     elif command in {"/help", "/contact"}:
2120         await _send_help(client, int(chat_id))
2121     else:
2122         await _handle_unknown_text(client, int(chat_id))
2123
2124
2125 async def _send_daily_report(client: httpx.AsyncClient, chat_id: int) -> None:
2126     """Kunlik hisobot: bugungi tasdiqlangan xaridlar, tushum, kun davomidagi
2127     rad etilgan cheklar, yangi talabalar va referral cashback kiritilari."""
2128     store = get_store()
2129     try:
2130         purchases = await store.list_purchases(limit=200)
2131     except Exception:
2132         purchases = []
2133     today = _dt.date.today().isoformat()
2134     todays = [p for p in purchases if str(p.get("created_at") or "").startswith(today)]
2135     approved = [p for p in todays if p.get("status") == "approved"]
2136     rejected = [p for p in todays if p.get("status") == "rejected"]
2137     pending = [p for p in todays if p.get("status") == "pending_approval"]
2138     revenue = sum(int(p.get("amount") or 0) for p in approved)
2139
2140     from app.services import wallet as wallet_service
2141     wallets_raw = await wallet_service._get_json(store, "wallets", {})
2142     todays_cashback = 0
2143     for w in wallets_raw.values():
2144         for e in (w.get("history") or []):
2145             if e.get("type") == "earn_referral" and str(e.get("created_at") or "").startswith(today):
2146                 try:
2147                     todays_cashback += int(e.get("delta") or 0)
2148                 except (TypeError, ValueError):
2149                     pass
2150
2151     try:
2152         total_users = await store.count_users()
2153     except Exception:
2154         total_users = 0
2155
2156     lines = [
2157         f"<b>👤 Kunlik hisobot – {today}</b>",
2158         "",
2159         f"🔥 <b>Bugungi tushum:</b> {_uzs(revenue)}",
2160         f"✅ <b>Tasdiqlangan xaridlar:</b> <b>{len(approved)} ta</b>",
2161         f"📄 <b>Kutilayotgan cheklar:</b> <b>{len(pending)} ta</b>",
2162         f"❌ <b>Rad etilgan:</b> <b>{len(rejected)} ta</b>",
2163         f"💰 <b>Bugungi referral cashback:</b> <b>{_uzs(todays_cashback)}</b>",
2164         f"👥 <b>Jami foydalanuvchilar:</b> <b>{total_users}</b>",
2165     ]
2166
2167     if approved:
2168         lines.append("")
2169         lines.append("<b>Xaridlar:</b>")
2170         for p in approved[:10]:
2171             lines.append(
2172                 f"• {_escape(p.get('student_name') or 'Talaba')} – "
2173                 f"{_escape(p.get('course_title') or 'Kurs')} ({_uzs(p.get('amount'))})"
2174             )
2175         if len(approved) > 10:
2176             lines.append(f"<i>...va yana {len(approved) - 10} ta</i>")
2177     await send_tg_message(
2178         client, chat_id, "\n".join(lines),
2179         {"inline_keyboard": [{"text": "🔗 Admin Panel", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}]}),
2180     )
2181
2182 async def _send_backup(client: httpx.AsyncClient, chat_id: int) -> None:
2183     """To'liq zaxira: barcha jadvallar va sozlamalar bitta JSON fayl sifatida
2184     admin chatga yuboriladi (Document)."""
2185     store = get_store()
2186     backup: Dict[str, Any] = {
2187         "_meta": {
2188             "created_at": _dt.datetime.now(_dt.timezone.utc).isoformat(),
2189             "storage": getattr(store, "backend_name", "?"),
2190             "version": "2.0.0",
2191         }
2192     }
2193     try:
2194         backup["users"] = await store.list_users(limit=1000)

```

```

2195 except Exception:
2196     backup["users"] = []
2197 try:
2198     backup["courses"] = await store.list_courses(published_only=False)
2199 except Exception:
2200     backup["courses"] = []
2201 try:
2202     backup["purchases"] = await store.list_purchases(limit=1000)
2203 except Exception:
2204     backup["purchases"] = []
2205 try:
2206     from app.api.banners import load_banners
2207     backup["banners"] = await load_banners()
2208 except Exception:
2209     backup["banners"] = []
2210 from app.services import wallet as wallet_service
2211 for key in ("wallets", "promo_codes", "referral_codes", "referral_links",
2212            "referral_rewarded", "referral_milestones_granted", "referral_settings"):
2213     try:
2214         raw = await store.get_setting(key)
2215         backup[key] = json.loads(raw) if raw else {}
2216     except Exception:
2217         backup[key] = {}
2218
2219 # /api/... URL larni R2 public URL ga almashtiramiz (tiklash uchun to'liqroq)
2220 payload = json.dumps(backup, ensure_ascii=False, default=str).encode("utf-8")
2221 files = {"document": (f"backup_{dt.date.today().isoformat()}.json", payload, "application/json")}
2222 try:
2223     response = await client.post(
2224         f"{API_URL}/sendDocument",
2225         data={"chat_id": str(chat_id), "caption": "📁 To'liq platforma xaxirasi (JSON)"},
2226         files=files,
2227     )
2228     if response.status_code != 200:
2229         logger.warning("Backup yuborilmadi: %s", response.text[:200])
2230 except httpx.HTTPError as exc:
2231     logger.error("Backup yuborishda xato: %s", exc)
2232
2233
2234 async def _daily_reports_loop(client: httpx.AsyncClient) -> None:
2235     """Har kuni soat 21:00 da barcha superadminlarga kunlik hisobot (uchishda)."""
2236     sent_for: str = ""
2237     while True:
2238         try:
2239             now = dt.datetime.now()
2240             target = now.replace(hour=21, minute=0, second=0, microsecond=0)
2241             if now >= target:
2242                 target += dt.timedelta(days=1)
2243                 await asyncio.sleep(min(max((target - now).total_seconds(), 30), 26 * 3600))
2244                 # 'bugun' hisobini uyg'ongandan keyin olamiz – hisobot aniq bo'ladi
2245                 today = dt.date.today().isoformat()
2246                 if sent_for != today:
2247                     sent_for = today
2248                     for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
2249                         await _send_daily_report(client, int(admin_id))
2250             except asyncio.CancelledError:
2251                 break
2252         except Exception:
2253             logger.exception("Kunlik hisobot tsiklida xato")
2254             await asyncio.sleep(600)
2255
2256
2257 async def start_telegram_bot_polling() -> None:
2258     """FastAPI lifespan ichida ishlaydigan, xatolarga chidamli long-polling xizmati."""
2259     if not "[REDACTED_BOT_TOKEN]":
2260         logger.warning("BOT_TOKEN sozlanmagan, Telegram polling ishga tushmadi.")
2261         return
2262
2263     logger.info("Course Academy Telegram bot polling ishga tushmoqda.")
2264     offset = 0
2265     allowed_updates = ["message", "callback_query", "chat_join_request", "my_chat_member"]
2266     async with httpx.AsyncClient(timeout=35.0) as client:
2267         # Kunlik 21:00 hisoboti – alohida fon vazifasi (pollingni bloklamaydi)
2268         reports_task = asyncio.create_task(_daily_reports_loop(client))
2269         await _telegram_call(
2270             client,
2271             "setMyCommands",
2272             {"commands": [
2273                 {"command": "start", "description": "Platformani ochish"},
2274                 {"command": "kurslar", "description": "Kurslar katalogi"},
2275                 {"command": "tolov", "description": "To'lov rekvizitlari"},
2276                 {"command": "help", "description": "Yordam markazi"},
2277                 {"command": "admin", "description": "Superadmin statistikasi"},
2278                 {"command": "hisobot", "description": "Kunlik hisobot (admin)"},
2279                 {"command": "backup", "description": "To'liq xaxira olish (admin)"},
2280             ]},
2281         )
2282         await _telegram_call(

```

```

2283     client,
2284     "setChatMenuButton",
2285     {"menu_button": {"type": "web_app", "text": "🎓 Course Academy", "web_app": {"url": settings.WEBAPP_URL}}},
2286 )
2287
2288 async def _safe_handle_tg_update(item: Dict[str, Any]) -> None:
2289     try:
2290         await handle_tg_update(client, item)
2291     except Exception:
2292         logger.exception("Telegram update ishlovida kutilmagan xato (update_id=%s)", item.get("update_id"))
2293
2294 while True:
2295     try:
2296         response = await client.get(
2297             f"{API_URL}/getUpdates",
2298             params={"offset": offset, "timeout": 25, "allowed_updates": json.dumps(allowed_updates)},
2299         )
2300         data = response.json()
2301         if response.status_code != 200 or not data.get("ok"):
2302             if response.status_code == 409:
2303                 logger.warning("Boshqa polling instansiyasi ishlayapti; qayta uriniladi.")
2304                 await asyncio.sleep(8)
2305             else:
2306                 logger.warning("getUpdates xatosi: %s", response.text[:300])
2307                 await asyncio.sleep(3)
2308             continue
2309         for item in data.get("result", []):
2310             offset = int(item["update_id"]) + 1
2311             asyncio.create_task(_safe_handle_tg_update(item))
2312     except asyncio.CancelledError:
2313         logger.info("Telegram polling to'xtatildi.")
2314         reports_task.cancel()
2315         try:
2316             await reports_task
2317         except (asyncio.CancelledError, Exception):
2318             pass
2319         break
2320     except (httpx.HTTPError, ValueError) as exc:
2321         logger.error("Telegram polling xatosi: %s", exc)
2322         await asyncio.sleep(3)

```